

Security in Applications

Silvia Llorente
Jaime Delgado

DAC - UPC

Security in applications

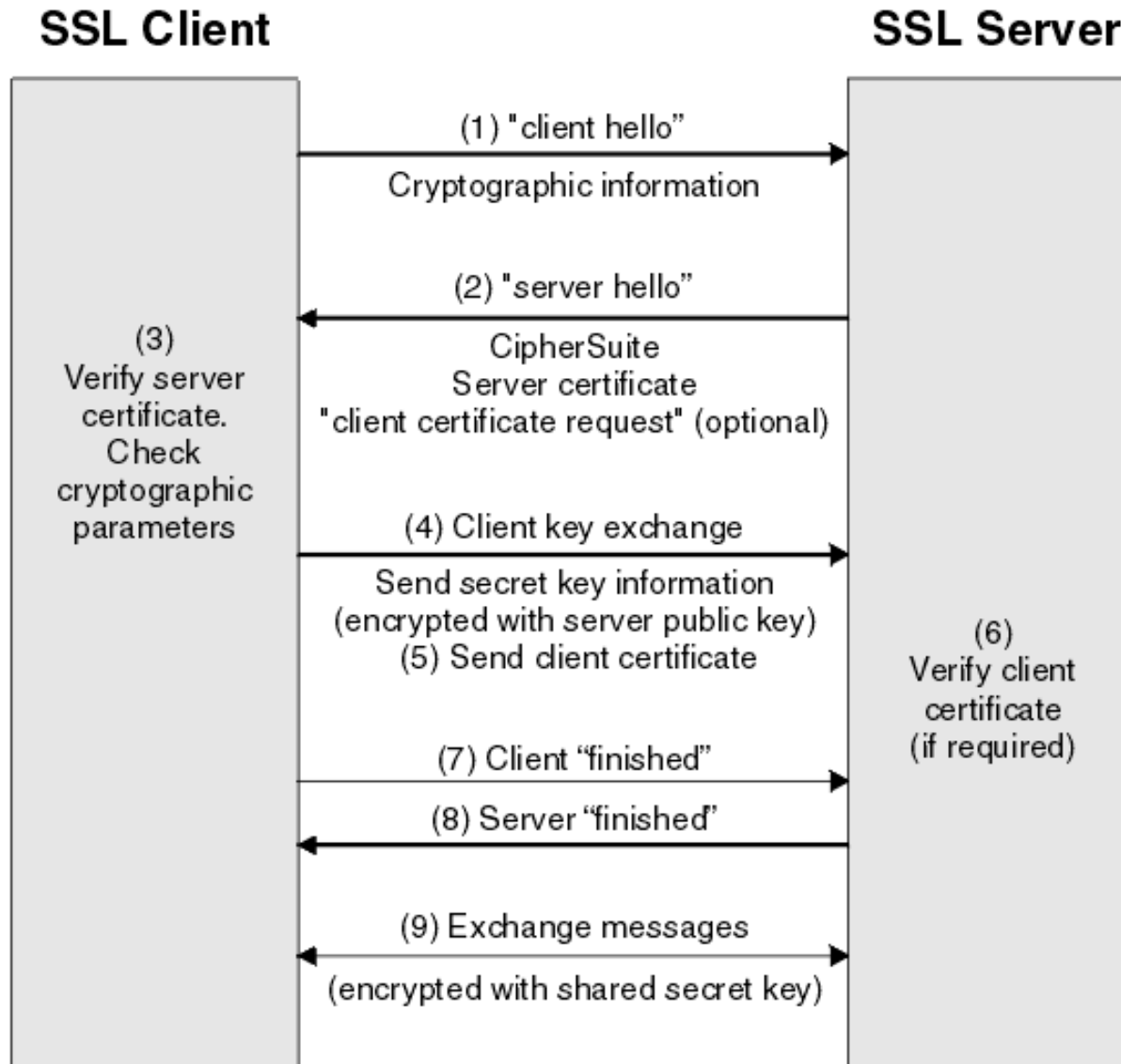
- **Security? Why do we need security?**
- <https://cybermap.kaspersky.com/es>

Security in applications

- **1.** Protocols:
 - Web (HTTP)
 - E-mail (content)
- **2.** Formats:
 - XML: XML Encryption, XML Signature
- **3.** Protocols specific for security: SAML, OAuth, JWT
- Others:
 - **4.** Data protection / Privacy / Digital identity
 - **5.** Intellectual rights for multimedia content

- Web (HTTP):
 - HTTPS: Secure transport (TCP) connection.
- E-mail:
 - S/MIME (Secure / Multipurpose Internet Mail Extensions)
 - MIME over pkcs#7 (enveloped). (PKCS: Public Key Cryptography Standards)
 - PGP (Pretty Good Privacy): Signature

HTTPS: SSL handshake (old, but still working)



SSL:
Secure Sockets
Layer

TLS (Transport Layer Security) Protocol

- Versions: Current TLSv1.2; New **TLSv1.3** (2017)
- *Handshake protocol phase*:
 - Authentication one or both sides (usually the server)
 - Negotiation: “cipher suites” (only Elliptic Curve *Diffie-Hellman* key exchange algorithms)
- *Record protocol phase*:
 - Carries and encapsulates data.
 - Adds a MAC (Message Authentication Code), encrypts *application protocol data* and adds a TLS header (5 bytes).

Bibliography: https://owasp.org/www-chapter-london/assets/slides/OWASPLondon20180125_TLSv1.3_Andy_Brodie.pdf

HTTPS: TLSv1.3 handshake & record

Client 

ClientHello
Supported cipher suites
Key Share

Finished
Application data 

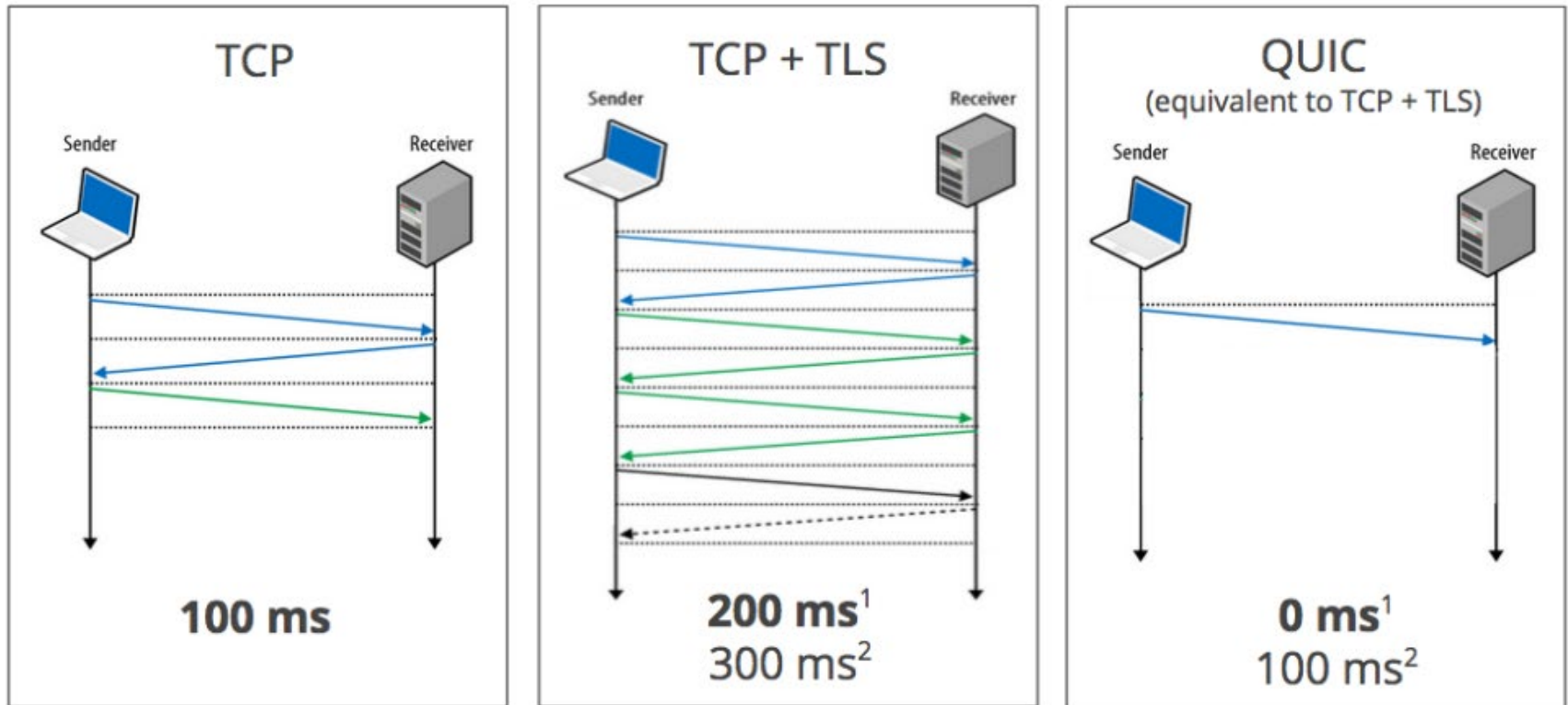
Server

ServerHello
Chosen cipher suite
Key share
Certificate
Finished

 Application data

HTTP3 and QUIC

Zero RTT Connection Establishment

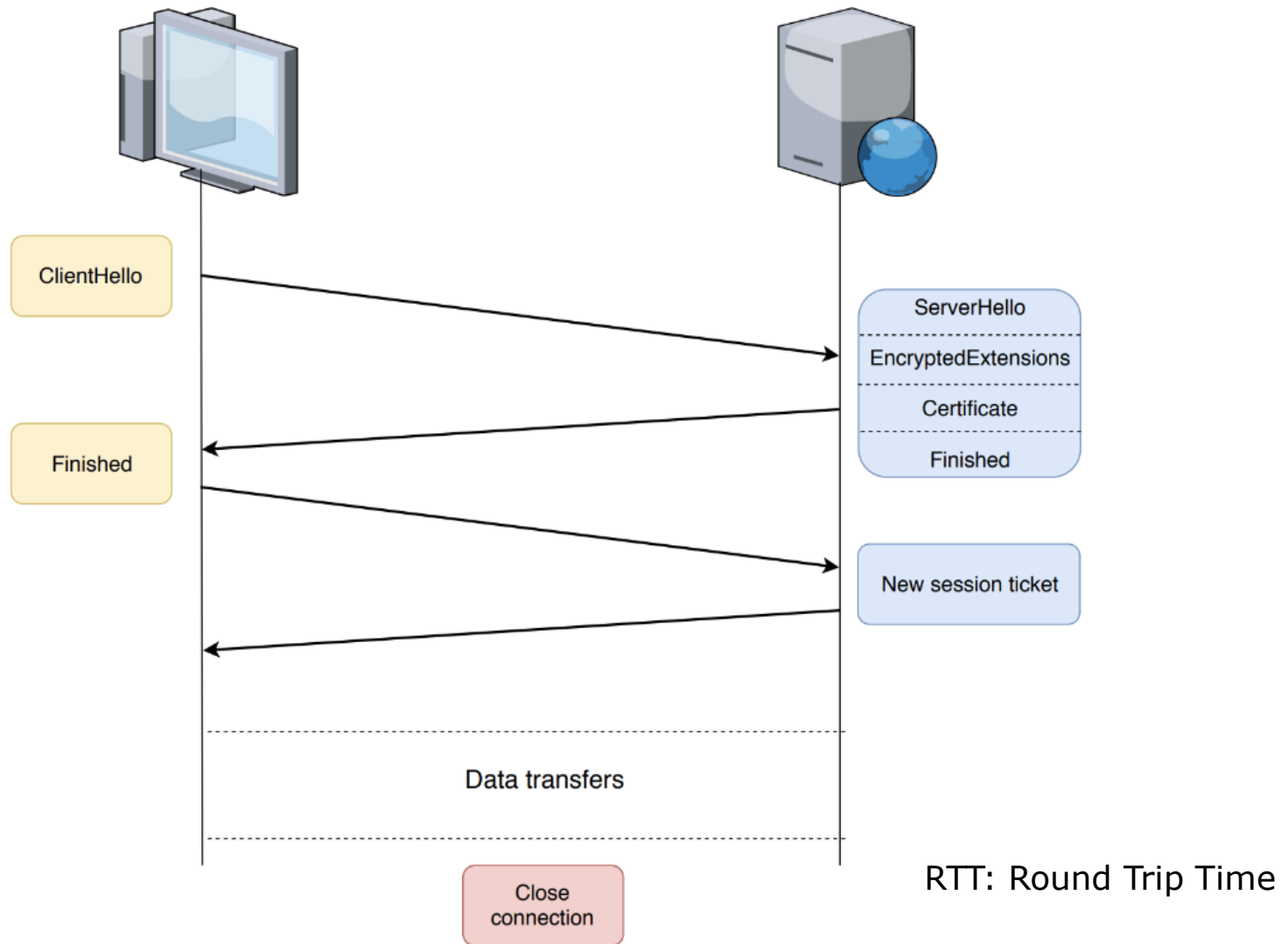


1. Repeat connection
2. Never talked to server before

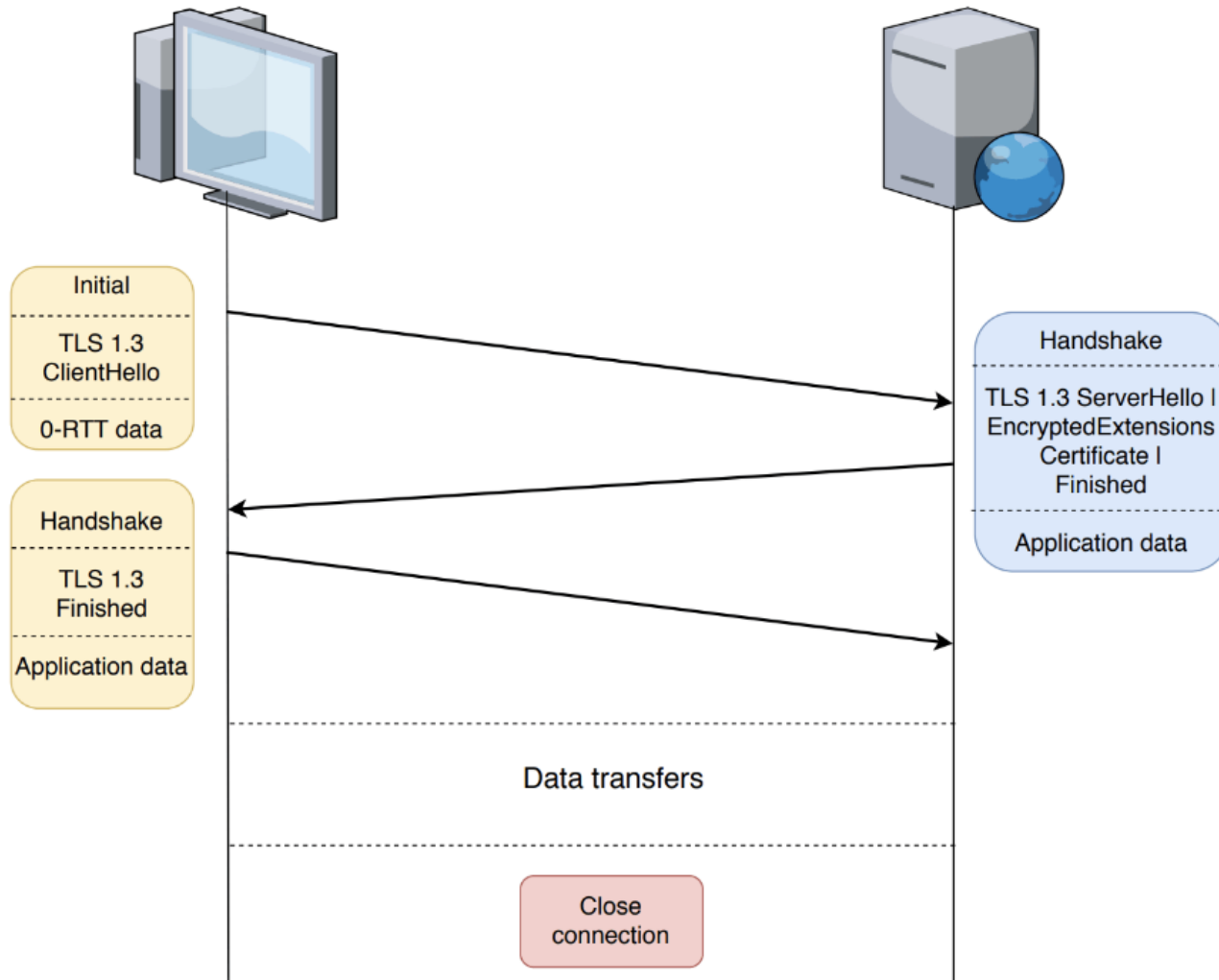
Handshakes on TCP, TCP+TLS and QUIC

Source: <https://ahorasomos.izertis.com/solidgear/quic-lo-que-se-esconde-detras-de-http-3/>
Bibliography: <https://datatracker.ietf.org/doc/html/rfc9114>

HTTP3 and QUIC



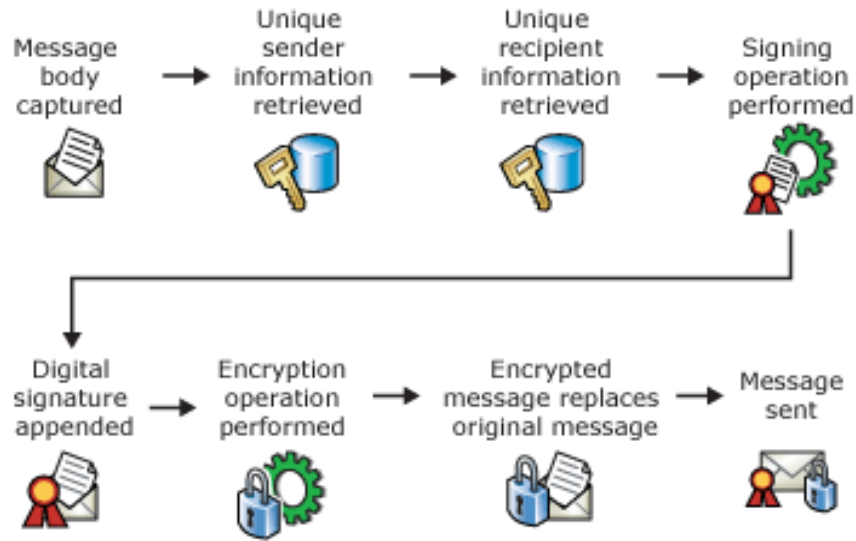
HTTP3 and QUIC



QUIC Handshake 0-RTT

S/MIME: Encrypt and sign message

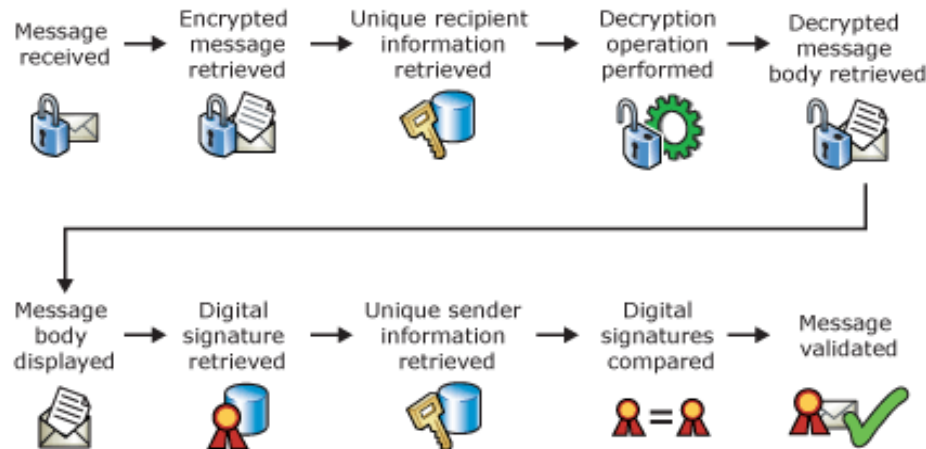
Digital signing and encrypting of an e-mail message



1. Message is captured.
 2. Information uniquely identifying the sender is retrieved.
 3. Information uniquely identifying the recipient is retrieved.
 4. Signing operation is performed on a message using the sender's unique information to produce a digital signature.
 5. Digital signature is appended to the message.
 6. Encryption operation is performed on the message using the recipient's information to produce an encrypted message.
 7. Original message is replaced by encrypted message.
 8. Message is sent.
- Source: [https://docs.microsoft.com/en-us/previous-versions/tn-archive/aa995740\(v=exchg.65\)](https://docs.microsoft.com/en-us/previous-versions/tn-archive/aa995740(v=exchg.65))

S/MIME: Decrypt message and verify signature

Decrypting an e-mail message and verifying a digital signature



1. Message is received.
2. Encrypted message is retrieved.
3. Information uniquely identifying the recipient is retrieved.
4. Decryption operation is performed on the encrypted message using the recipient's unique information to produce an unencrypted message.
5. Unencrypted message is returned.
6. Unencrypted message is returned to the recipient.
7. Digital signature is retrieved from the unencrypted message.
8. Information identifying the sender is retrieved.
9. Signing operation is performed on the unencrypted message using the sender's information to produce a digital signature.
10. Digital signature included with the message is compared against the digital signature produced on receipt.
11. If the digital signatures match, the message is valid.

Security in applications

- **1.** Protocols:
 - Web (HTTP)
 - E-mail (content)
- **2. Formats:**
 - **XML: XML Encryption, XML Signature**
- **3.** Protocols specific for security: SAML, OAuth, JWT
- Others:
 - **4.** Data protection / Privacy / Digital identity
 - **5.** Intellectual rights for multimedia content

- XML-based security
 - XML Encryption
 - XML Signature

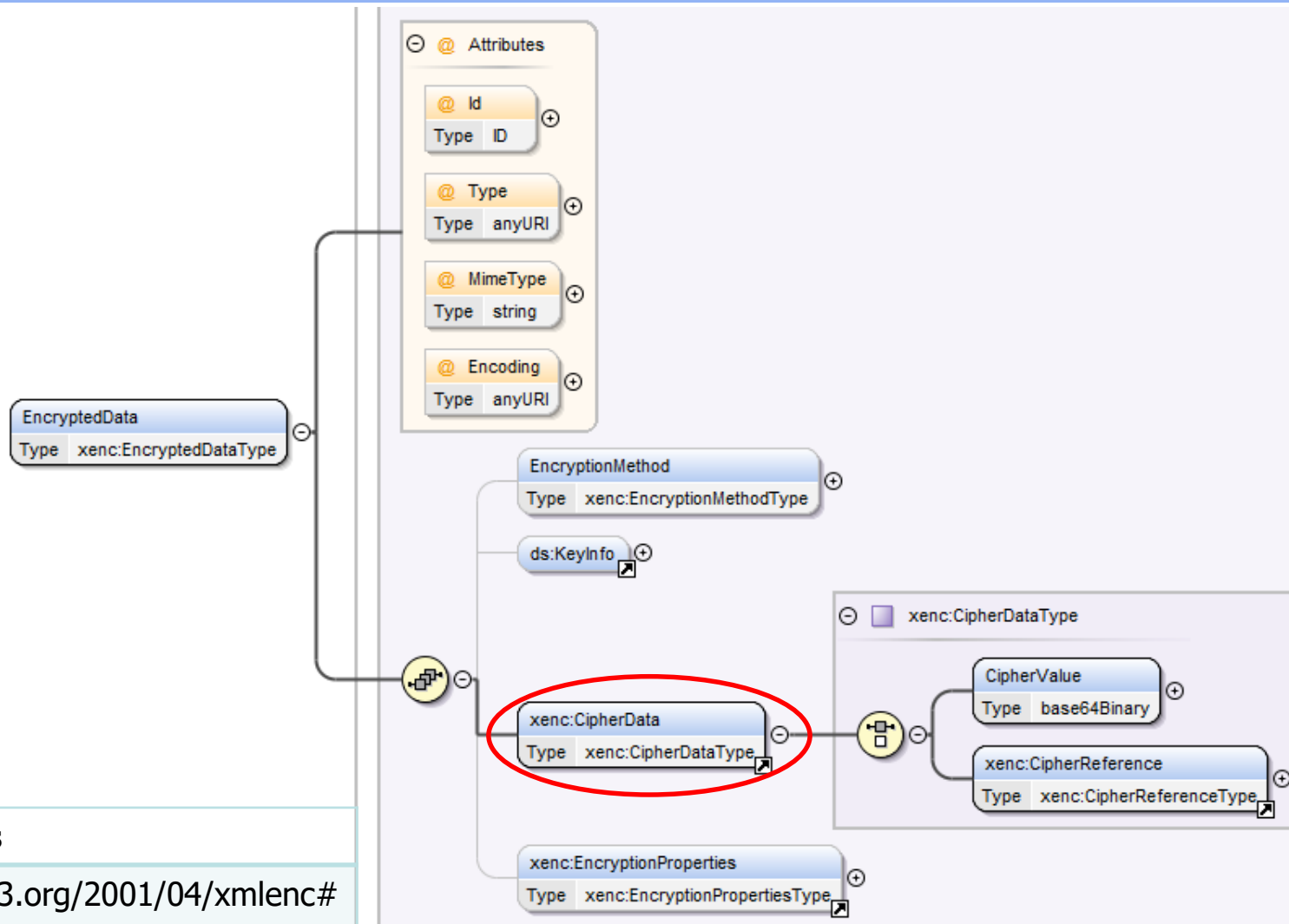
XML Encryption

- W3C Recommendation (*11th April 2013*)
 - XML Encryption Syntax and Processing Version 1.1
<https://www.w3.org/TR/xmlenc-core1/>
- Specifies a process to encrypt data:
 - Arbitrary data (including XML documents)
 - XML element
 - Content of an XML element
- The result is represented in XML:
 - Element EncryptedData (XML Encryption)
 - Contains the ciphered data (in one of its child elements)
 - References the ciphered data (through an URI reference)

Encrypting process

- Two alternatives:
 - XML element or its content is encrypted
 - EncryptedData element replaces it in the encrypted version (rest remains equal)
 - Complete document
 - EncryptedData element will be the root element of the encrypted XML document

EncryptedData element structure

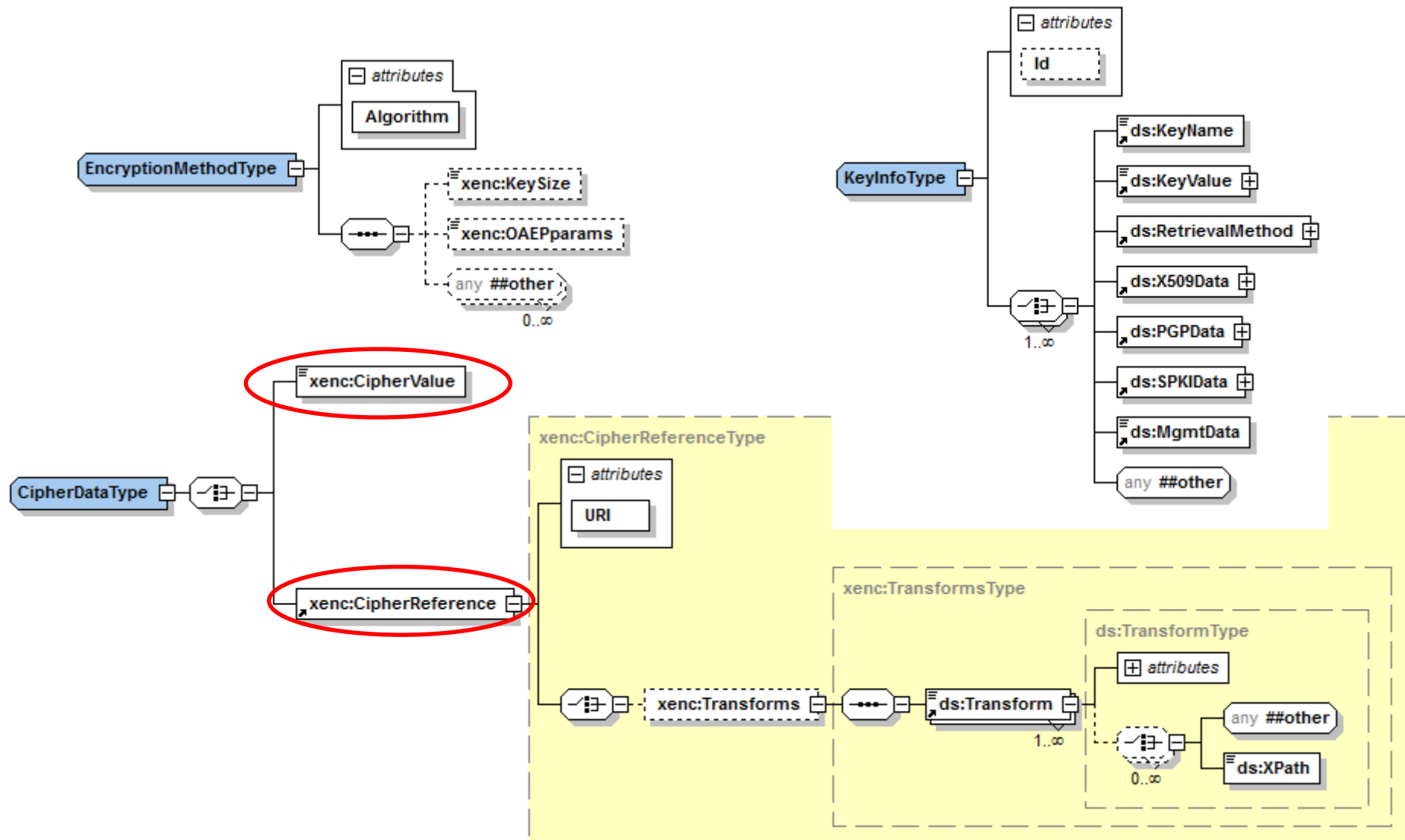


Prefix	Namespaces
xenc	http://www.w3.org/2001/04/xmlenc#
xenc11	http://www.w3.org/2009/xmlenc11#

Encrypting process

1. Select algorithm (and parameters) to use.
Element `xenc:encryptionMethod`
2. Obtain the key. Element `ds:keyInfo`
3. Encrypt the data (octets result of serializing data in UTF-8)
4. Build the structure `xenc:EncryptedData`
(for the key `xenc:EncryptedKey`)
 - Two alternatives
 - i. Ciphered data included: element `xenc:CipherValue` with the sequence of encrypted octets coded in base 64
 - ii. External ciphered data: element `xenc:CipherReference` with the URI and transforms (if any)

Element types – EncryptedData



XML encryption: Original file

```
<?xml version='1.0'?>  
<PaymentInfo  
  xmlns="http://example.org/paymentv2">  
  <Name>John Smith</Name>  
  <CreditCard Limit="5000" Currency="USD">  
    <Number>4019 2445 0277 5567</Number>  
    <Issuer>Example Bank</Issuer>  
    <Expiration>04/24</Expiration>  
  </CreditCard>  
</PaymentInfo>
```

XML encryption: Info to encrypt

```
<?xml version='1.0'?>  
<PaymentInfo  
  xmlns="http://example.org/paymentv2">  
  <Name>John Smith</Name>  
  <CreditCard Limit="5000" Currency="USD">  
    <Number>4019 2445 0277 5567</Number>  
    <Issuer>Example Bank</Issuer>  
    <Expiration>04/24</Expiration>  
  </CreditCard>  
</PaymentInfo>
```

XML encryption: Info encrypted

```
<?xml version='1.0'?>
<PaymentInfo
  xmlns="http://example.org/paymentv2">
  <Name>John Smith</Name>
  <EncryptedData
    xmlns="http://www.w3.org/2001/04/xmlenc#"
    Type="http://www.w3.org/2001/04/xmlenc#Element">
    <CipherData>
      <CipherValue>PENyZWRpd</CipherValue>
    </CipherData>
  </EncryptedData>
</PaymentInfo>
```

XML encryption: Value to encrypt

```
<?xml version='1.0'?>
<PaymentInfo
  xmlns="http://example.org/paymentv2">
  <Name>John Smith</Name>
  <CreditCard Limit="5000" Currency="USD">
    <Number>4019 2445 0277 5567</Number>
    <Issuer>Example Bank</Issuer>
    <Expiration>04/24</Expiration>
  </CreditCard>
</PaymentInfo>
```

XML encryption: Encrypted info

```
<?xml version='1.0'?>
<PaymentInfo xmlns="http://example.org/paymentv2">
  <Name>John Smith</Name>
  <CreditCard Limit="5000" Currency="USD">
    <Number>
      <EncryptedData
        xmlns="http://www.w3.org/2001/04/xmlenc#"
        Type="http://www.w3.org/2001/04/xmlenc#Content">
        <CipherData>
          <CipherValue>NDaxOSayN</CipherValue>
        </CipherData>
      </EncryptedData>
    </Number>
    <Issuer>Example Bank</Issuer>
    <Expiration>04/24</Expiration>
  </CreditCard>
</PaymentInfo>
```


XML encryption: Info to encrypt

```
<?xml version='1.0'?>
```

```
<PaymentInfo
```

```
  xmlns="http://example.org/paymentv2">
```

```
    <Name>John Smith</Name>
```

```
    <CreditCard Limit="5000" Currency="USD">
```

```
      <Number>4019 2445 0277 5567</Number>
```

```
      <Issuer>Example Bank</Issuer>
```

```
      <Expiration>04/24</Expiration>
```

```
    </CreditCard>
```

```
</PaymentInfo>
```

XML encryption: Encrypted XML

```
<?xml version='1.0'?>
```

```
<EncryptedData
```

```
  xmlns="http://www.w3.org/2001/04/xmlenc#"
  MimeType="text/xml">
```

```
  <CipherData>
```

```
    <CipherValue>PFBheW1lb</CipherValue>
```

```
  </CipherData>
```

```
</EncryptedData>
```

XML Encryption: XML Schema versions

- Current version (coming from
<https://www.w3.org/TR/xmlenc-core1/>)
<https://www.w3.org/TR/xmlenc-core1/xenc-schema-11.xsd>
- Initial version (included by current one)
<http://www.w3.org/TR/2002/REC-xmlenc-core-20021210/xenc-schema.xsd>

- XML-based security
 - XML Encryption
 - **XML Signature**

Digital signature

- **Integrity:** If the signed information is modified, signature validation will fail
- **No repudiation:** The entity who signed the information cannot claim it did not sign
- **Authentication:** We know who signed, since we can validate it with its public key

XML Signature

- W3C Recommendation (*11th April 2013*)
 - XML Signature Syntax and Processing Version 1.1
<https://www.w3.org/TR/xmlsig-core1/>
- Defines a **XML syntax** for the **digital signature** that provides **integrity**, and messages and signatories **authentication**
- Used by different web technologies: SOAP, SAML, etc.

XML signatures

- *Over data external to the signature element:*
 - **Detached:** Used to sign a resource out of the XML document that contains the signature. Related through an URI
- *Over data within the same XML file as the signature:*
 - **Enveloped:** Used to sign a part of the document that contains the signature. The signature is an element of the XML content signed (signature not to include their own value in the calculation of the SignatureValue)
 - **Enveloping:** The XML signature contains the signed data inside itself. Related through fragment identifiers

Resources:

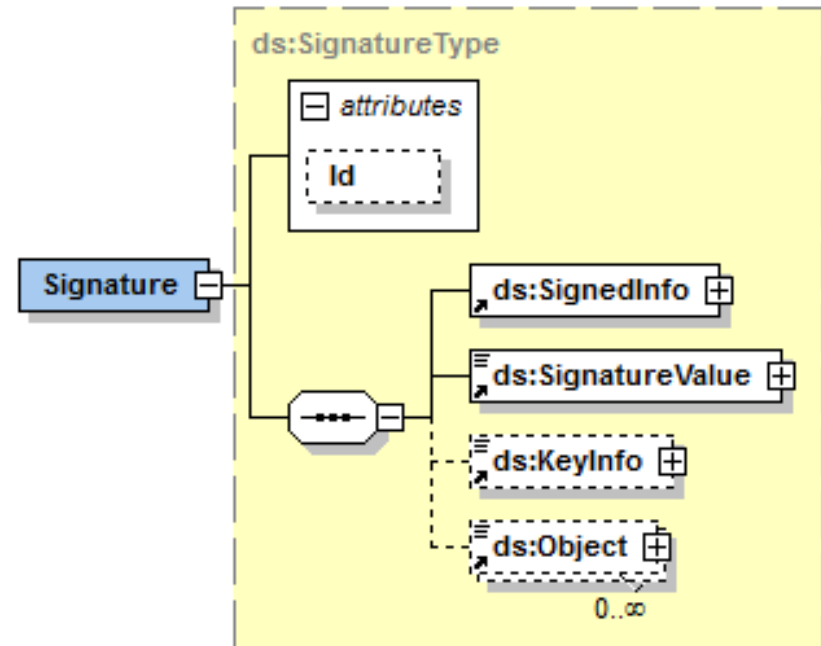
<https://cca.gov.in/sites/files/pdf/guidelines/CCA-XMLSP.pdf>

<https://ec.europa.eu/digital-building-blocks/DSS/webapp-demo/doc/dss-documentation.html>

<https://ec.europa.eu/digital-building-blocks/DSS/webapp-demo/apidocs/eu/europa/esig/dss/simplereport/jaxb/XmlSignature.html>

XML signature: Element to add

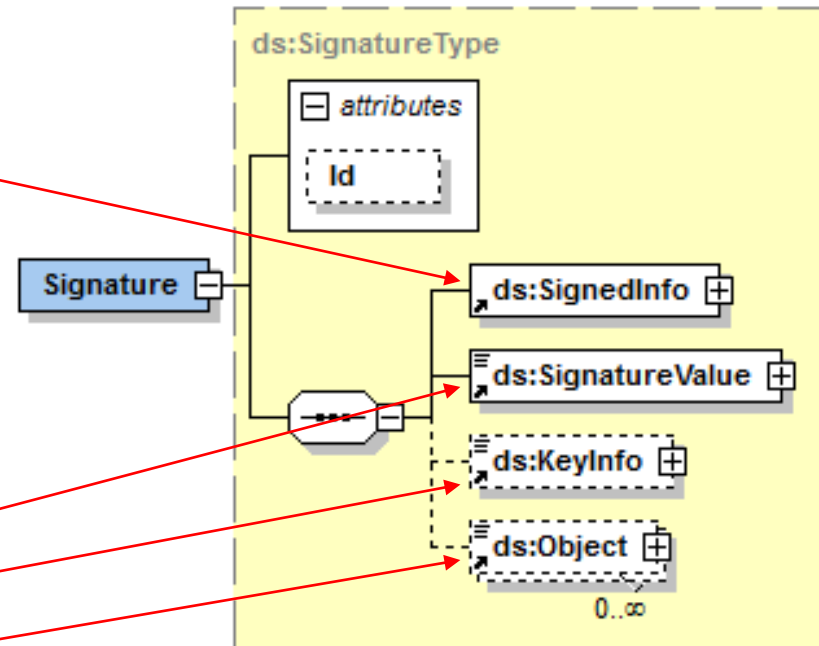
```
<Signature>
  <SignedInfo>
    <CanonicalizationMethod/>
    <SignatureMethod/>
    <Reference>
      <Transforms>
      <DigestMethod>
      <DigestValue>
    </Reference>
    <Reference/> ...
  </SignedInfo>
  <SignatureValue/>
  <KeyInfo/>
  <Object/>
</Signature>
```



Prefix	Namespace
ds	http://www.w3.org/2000/09/xmldsig#
dsig11	http://www.w3.org/2009/xmldsig11#

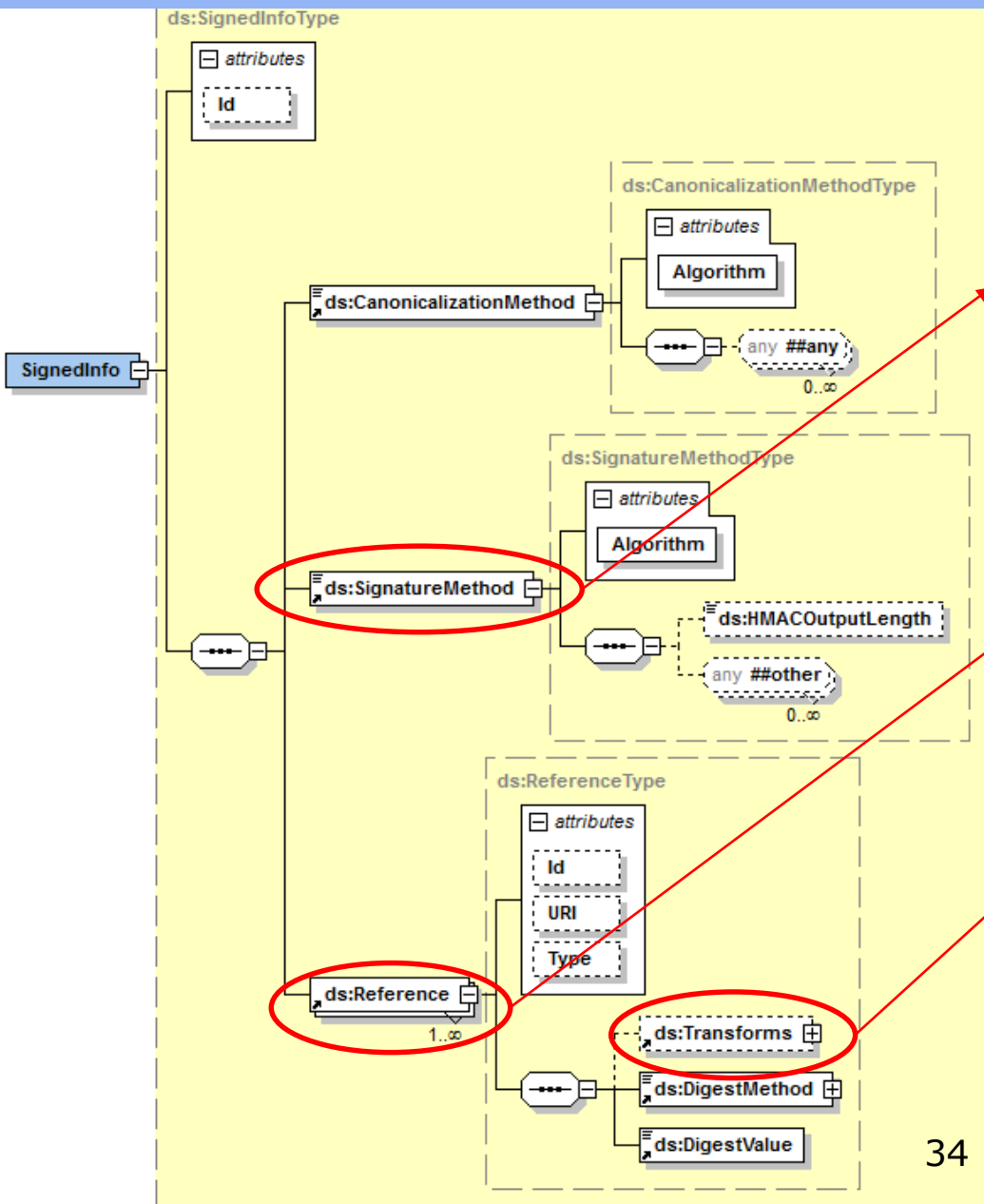
XML signature: Element to add

```
<Signature>
  <SignedInfo>
    <CanonicalizationMethod/>
    <SignatureMethod/>
    <Reference>
      <Transforms>
      <DigestMethod>
      <DigestValue>
    </Reference>
    <Reference/> ...
  </SignedInfo>
  <SignatureValue/>
  <KeyInfo/>
  <Object/>
</Signature>
```



Prefix	Namespace
ds	http://www.w3.org/2000/09/xmldsig#
dsig11	http://www.w3.org/2009/xmldsig11#

XML Signature Elements: *SignedInfo*



SignedInfo:

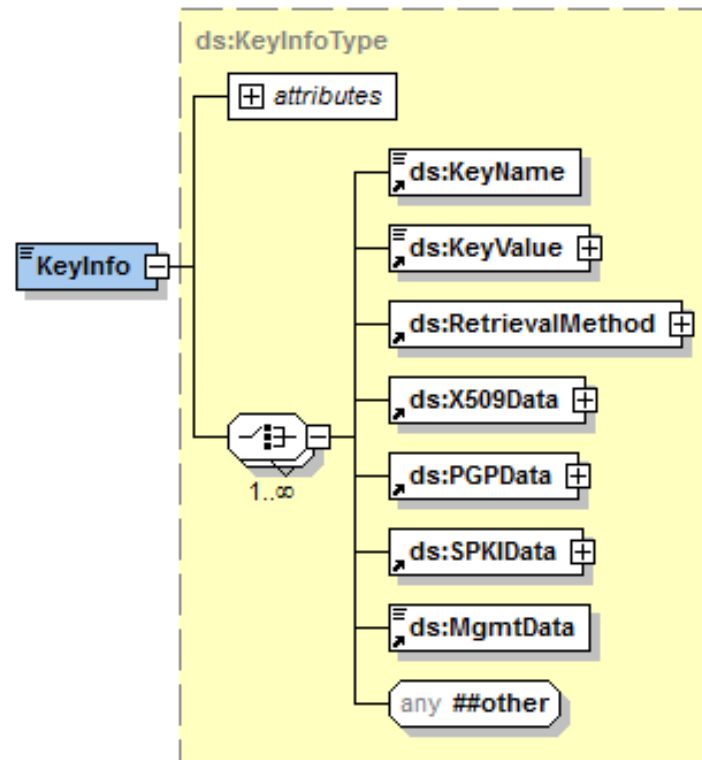
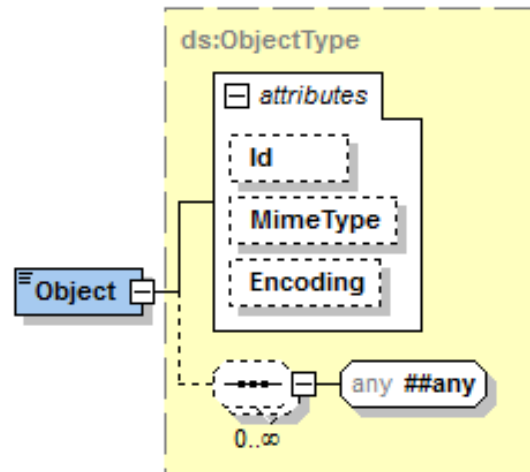
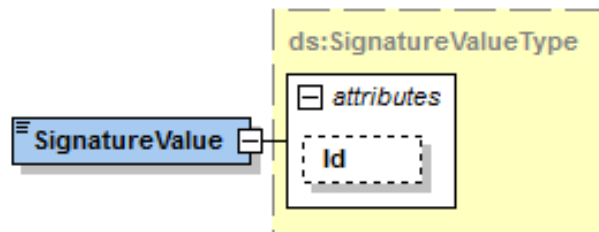
Information that is really signed

SignatureMethod: Algorithm used to obtain the value of the signature. Combination of a digest algorithm and a key algorithm, and possibly others, such as padding

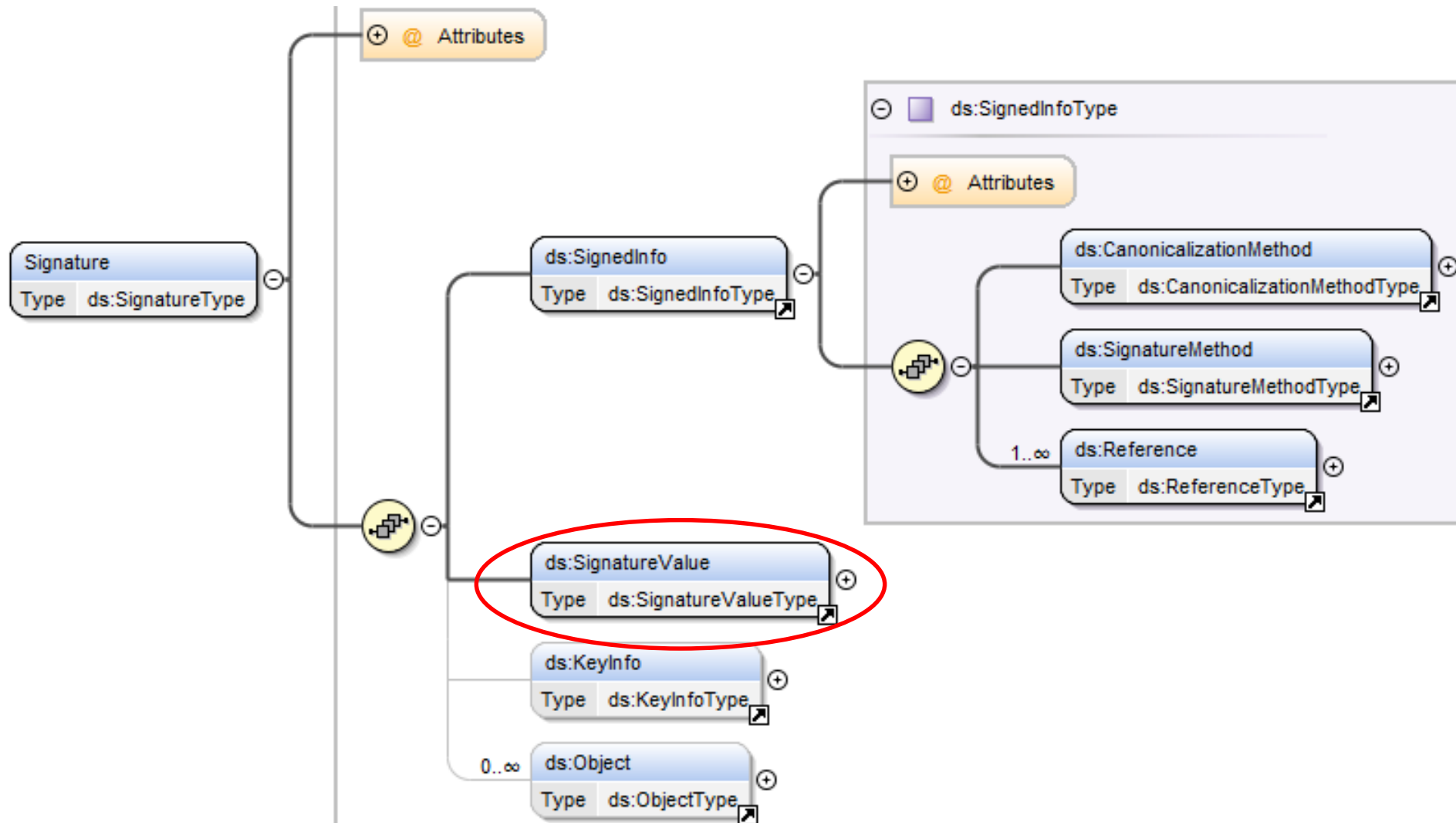
Reference: Includes the digest method and calculated value over the data object (URI attribute)

Transforms: Ordered list with the processing steps applied to the content before applying the digest algorithm (Ex.: canonicalization, XSLT, XPath, etc.)

XML Signature Elements: Others



Signature Element structure



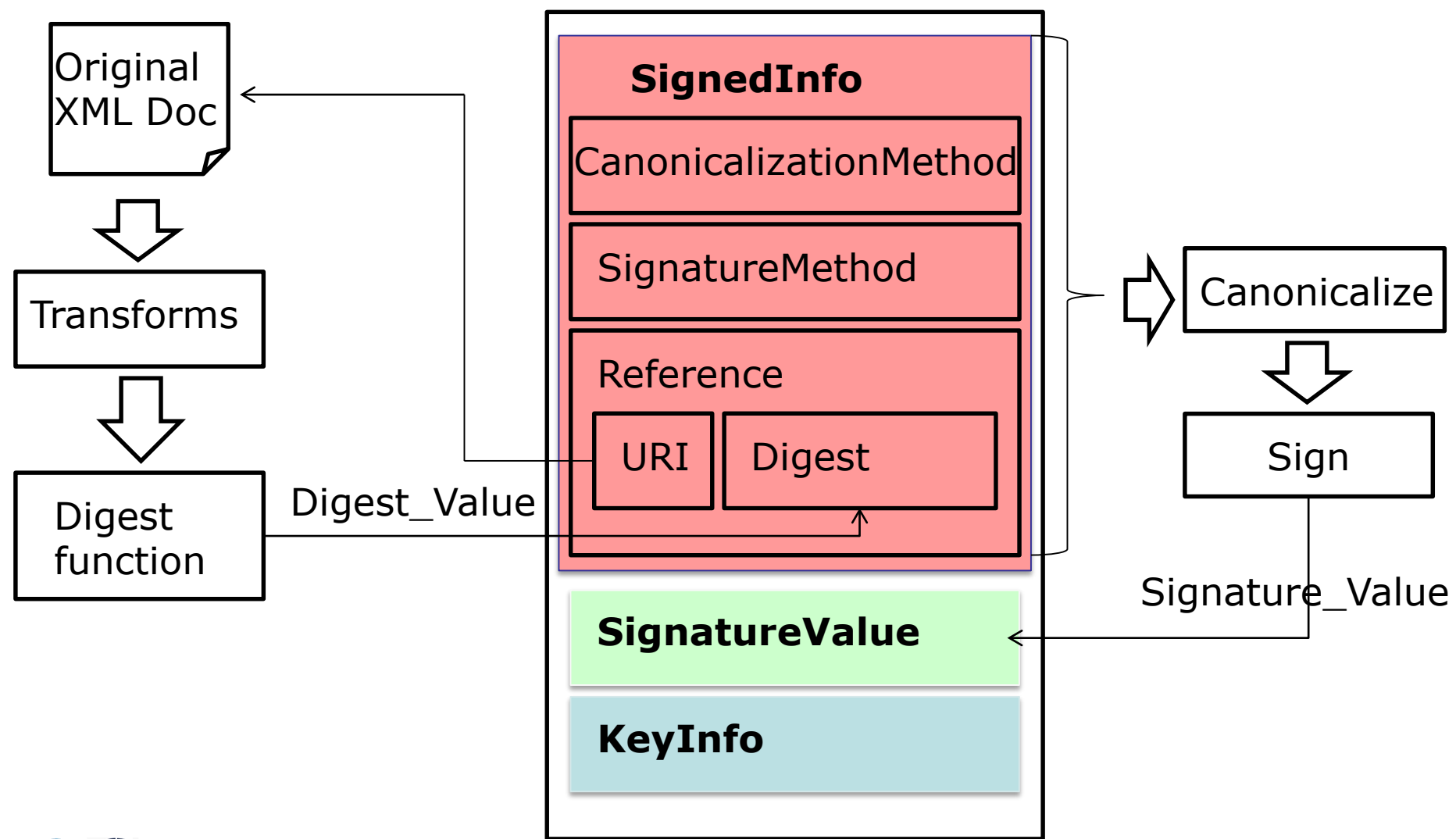
Example

```
<Signature Id="MyFirstSignature" xmlns="http://www.w3.org/2000/09/xmldsig#">
  <SignedInfo>
    <CanonicalizationMethod Algorithm="https://www.w3.org/TR/xml-c14n11/" />
    <SignatureMethod Algorithm="http://www.w3.org/2000/09/xmldsig#dsa-sha1" />
    <Reference URI="https://www.w3.org/TR/xhtml1/">
      <Transforms>
        <Transform Algorithm="https://www.w3.org/TR/xml-c14n11/" />
      </Transforms>
      <DigestMethod Algorithm="http://www.w3.org/2000/09/xmldsig#sha1" />
      <DigestValue>dGhpcyBpcyBub3QgYSBzaWduYXR1cmUK...</DigestValue>
    </Reference>
  </SignedInfo>
  <SignatureValue>...</SignatureValue>
  <KeyInfo>
    <KeyValue>
      <DSAKeyValue>
        <P>...</P><Q>...</Q><G>...</G><Y>...</Y>
      </DSAKeyValue>
    </KeyValue>
  </KeyInfo>
</Signature>
```

XML signature: XML Schema versions

- Current version
(<https://www.w3.org/TR/xmlsig-core1/>)
<https://www.w3.org/TR/xmlsig-core1/xmlsig11-schema.xsd>
- Initial version (included by current one)
<https://www.w3.org/TR/2002/REC-xmlsig-core-20020212/xmlsig-core-schema.xsd>

Model



Canonicalization steps (1)

- Character codification (UTF-8)
- Normalize end of line
- Normalize attribute values (include default values)
- Substitute entity references
- Deletion of XML declaration and DTD
- Empty elements: `<elem></elem>`
- Normalize blank spaces out of elements

Canonicalization steps (2)

- Unique delimiter for attribute values: "
- Special characters represented with numeric references (&#x...;)
- Delete unneeded namespaces
- Attribute order:
 - Namespaces, by lexicographical URI order
 - Attributes, by lexicographical name order

Security in applications

- **1.** Protocols:
 - Web (HTTP)
 - E-mail (content)
- **2.** Formats:
 - XML: XML Encryption, XML Signature
- **3. Protocols specific for security: SAML, OAuth, JWT**
- Others:
 - **4.** Data protection / Privacy / Digital identity
 - **5.** Intellectual rights for multimedia content

SAML (Security Assertion Markup Language)

- v2.0 March 2005. OASIS
(Organization for the Advancement of Structured Information Standards)
<https://www.oasis-open.org/standard/saml/>
- Interchange of authentication and authorization data
- Based in XML
- Security tokens: “Assertions” (user info)
- Interchange between an “identity provider” and a “service provider”
- Useful for authentication and authorization in web environments

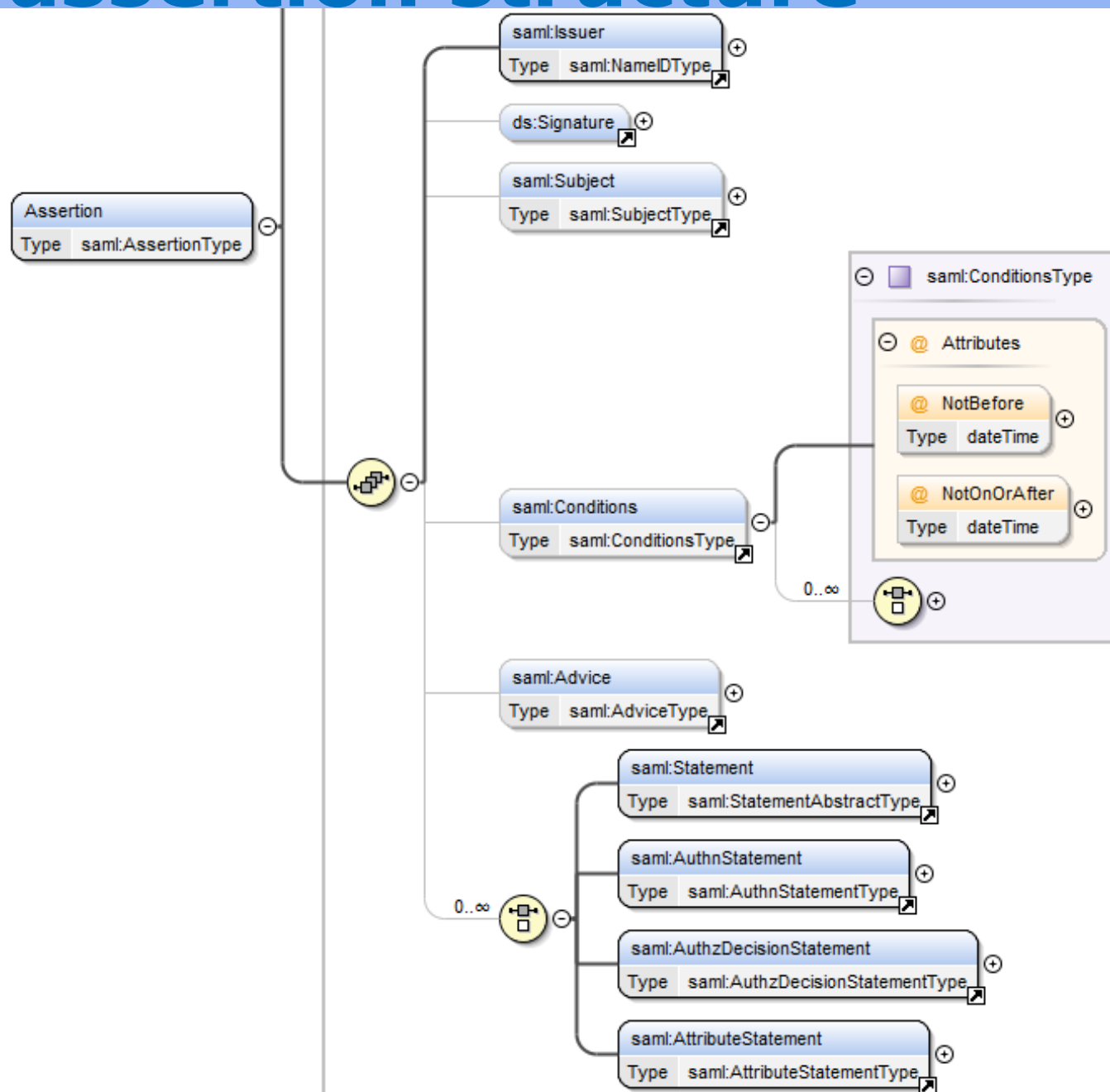
SAML (Security Assertion Markup Language)

- v2.0 March 2005. OASIS
(Organization for the Advancement of Structured Information Standards)
<https://www.oasis-open.org/standard/saml/>
- Interchange of authentication and authorization data
- Based in XML
- Security tokens: "Assertions" (user info)
- **Interchange between an "identity provider" and a "service provider"**
- Useful for authentication and authorization in web environments

SAML assertion structure

```
1: <saml:Assertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
2:   Version="2.0"
3:   IssueInstant="2005-01-31T12:00:00Z">
4:   <saml:Issuer Format=urn:oasis:names:SAML:2.0:nameid-format:entity>
5:     http://idp.example.org
6:   </saml:Issuer>
7:   <saml:Subject>
8:     <saml:NameID
9:       Format="urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress">
10:      j.doe@example.com
11:    </saml:NameID>
12:  </saml:Subject>
13:  <saml:Conditions
14:    NotBefore="2005-01-31T12:00:00Z"
15:    NotOnOrAfter="2005-01-31T12:10:00Z">
16:  </saml:Conditions>
17:  <saml:AuthnStatement
18:    AuthnInstant="2005-01-31T12:00:00Z" SessionIndex="67775277772">
19:    <saml:AuthnContext>
20:      <saml:AuthnContextClassRef>
21:        urn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport
22:      </saml:AuthnContextClassRef>
23:    </saml:AuthnContext>
24:  </saml:AuthnStatement>
25: </saml:Assertion>
```

SAML assertion structure



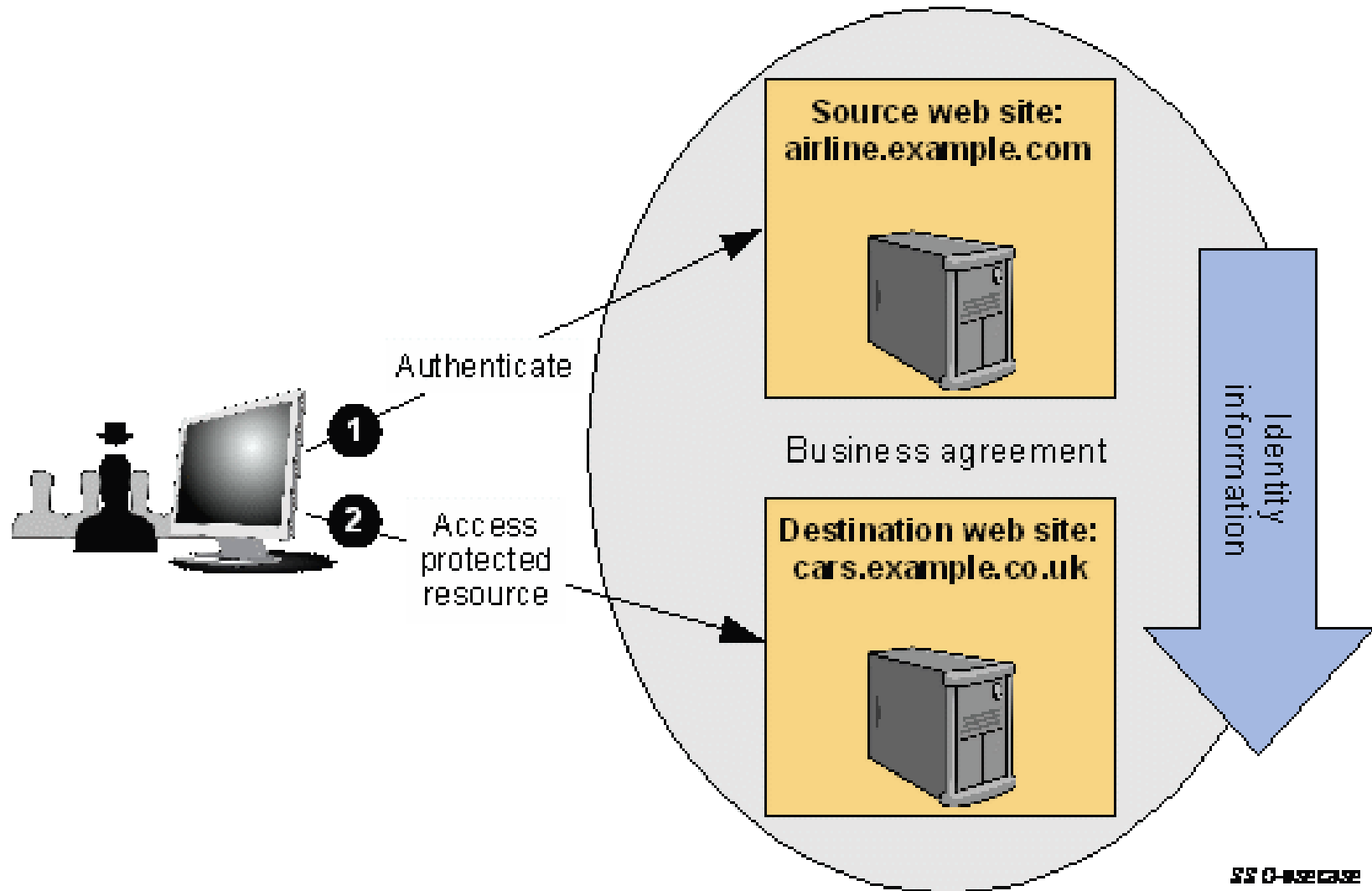
SAML attribute statement structure

```
1: <saml:AttributeStatement>
2:   <saml:Attribute
3:     xmlns:x500="urn:oasis:names:tc:SAML:2.0:profiles:attribute:X500"
4:     NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:uri"
5:     Name="urn:oid:2.5.4.42"
6:     FriendlyName="givenName">
7:     <saml:AttributeValue xsi:type="xs:string"
8:       x500:Encoding="LDAP">John</saml:AttributeValue>
9:   </saml:Attribute>
10:  <saml:Attribute
11:    NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:basic"
12:    Name="LastName">
13:    <saml:AttributeValue
14:      xsi:type="xs:string">Doe</saml:AttributeValue>
15:  </saml:Attribute>
16:  <saml:Attribute
17:    NameFormat="http://smithco.com/attr-formats"
18:    Name="CreditLimit">
19:    xmlns:smithco="http://www.smithco.com/smithco-schema.xsd"
20:    <saml:AttributeValue xsi:type="smithco:type">
21:      <smithco:amount currency="USD">500.00</smithco:amount>
22:    </saml:AttributeValue>
23:  </saml:Attribute>
24: </saml:AttributeStatement>
```

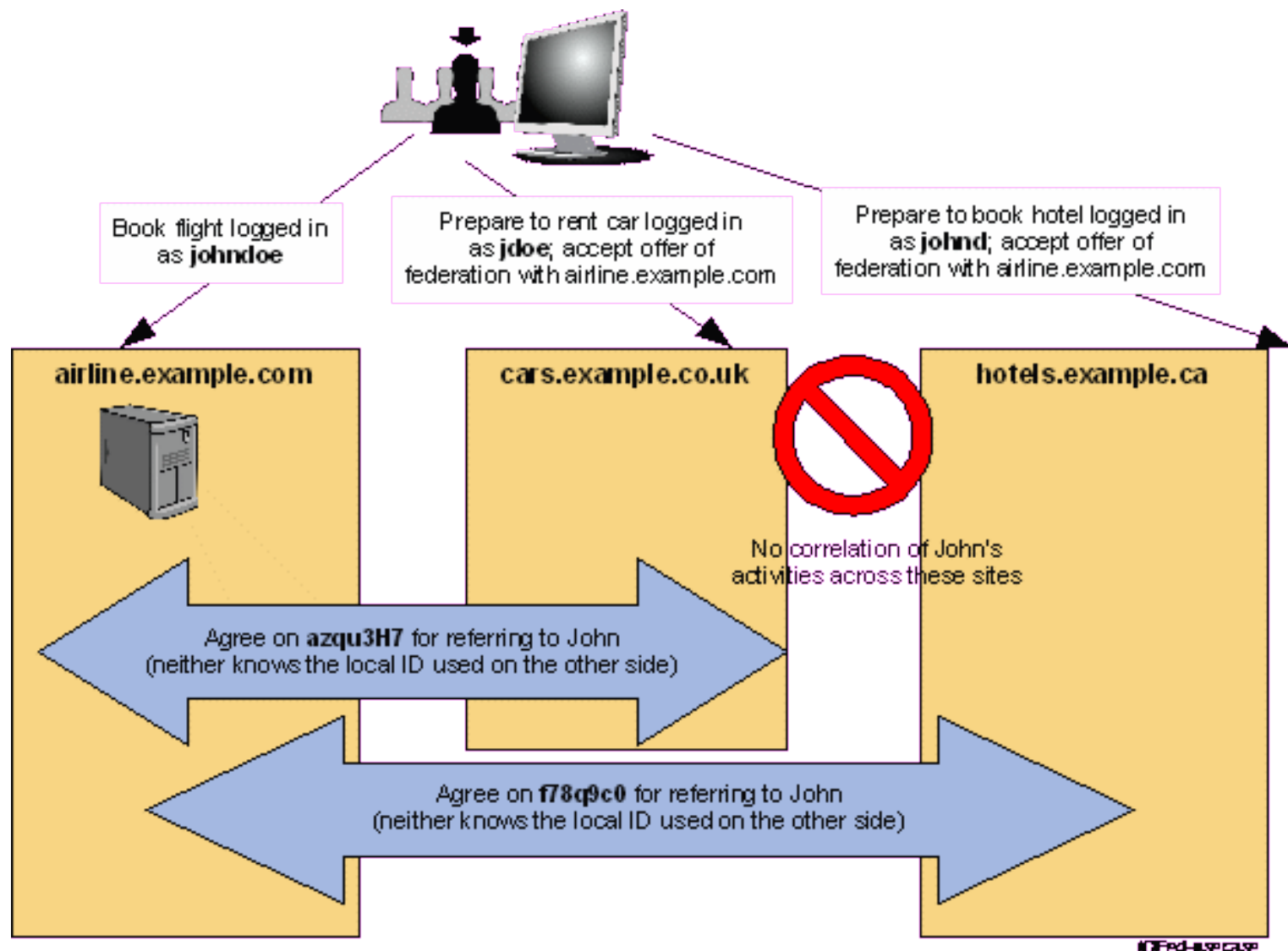
SAML: Examples of use

- **Single sign-on**
 - Standard browsers
 - Enhanced HTTP clients
- **Identity federation**
 - Using a well-known name or attribute
 - For anonymous users by means of attributes
 - Using a privacy-preserving pseudonym

Single Sign-On (SSO) use case



Identity Federation use case



NO Fed use case

SAML: SOAP binding example

```
1:  <?xml version="1.0" encoding="UTF-8"?>
2:  <env:Envelope
3:    xmlns:env="http://www.w3.org/2003/05/soap/envelope/">
4:    <env:Body>
5:      <samlp:AttributeQuery
6:        xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
7:        xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol"
8:        ID="aaf23196-1773-2113-474a-fell4412ab72"
9:        Version="2.0"
10:       IssueInstant="2006-07-17T20:31:40Z">
11:       <saml:Issuer>http://example.sp.com</saml:Issuer>
12:       <saml:Subject>
13:         <saml:NameID
14:           Format="urn:oasis:names:tc:SAML:1.1:nameid-format:X509SubjectName"
15:           C=US, O=NCSA-TEST, OU=User, CN=trscavo@uiuc.edu
16:         </saml:NameID>
17:       </saml:Subject>
18:       <saml:Attribute
19:         NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:uri"
20:         Name="urn:oid:2.5.4.42"
21:         FriendlyName="givenName">
22:       </saml:Attribute>
23:     </samlp:AttributeQuery>
24:   </env:Body>
25: </env:Envelope>
```

```
1:  <?xml version="1.0" encoding="UTF-8"?>
2:  <env:Envelope xmlns:env="http://schemas.xmlsoap.org/soap/envelope/">
3:    <env:Body>
4:      <samlp:Response
5:        xmlns:samlp="urn:oasis:names:tc:SAML:2.0:protocol"
6:        xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
7:        Version="2.0"
8:        ID="i92f8b5230dc04d73e93095719d191915fdc67d5e"
9:        IssueInstant="2006-07-17T20:31:41Z"
10:       InResponseTo="aaf23196-1773-2113-474a-fell4412ab72">
11:       <saml:Issuer>http://idp.example.org</saml:Issuer>
12:       <samlp:Status>
13:         <samlp:StatusCode Value="urn:oasis:names:tc:SAML:2.0:status:Success"/>
14:       </samlp:Status>
15:       ...SAML assertion...
16:     </samlp:Response>
17:   </env:Body>
18: </env:Envelope>
```

HTTP response

SOAP envelope

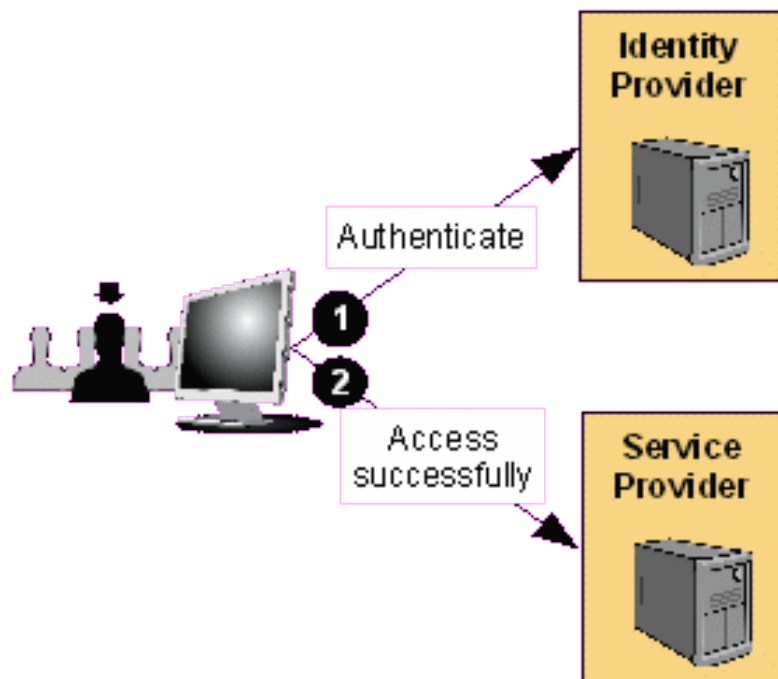
SOAP header

SOAP body

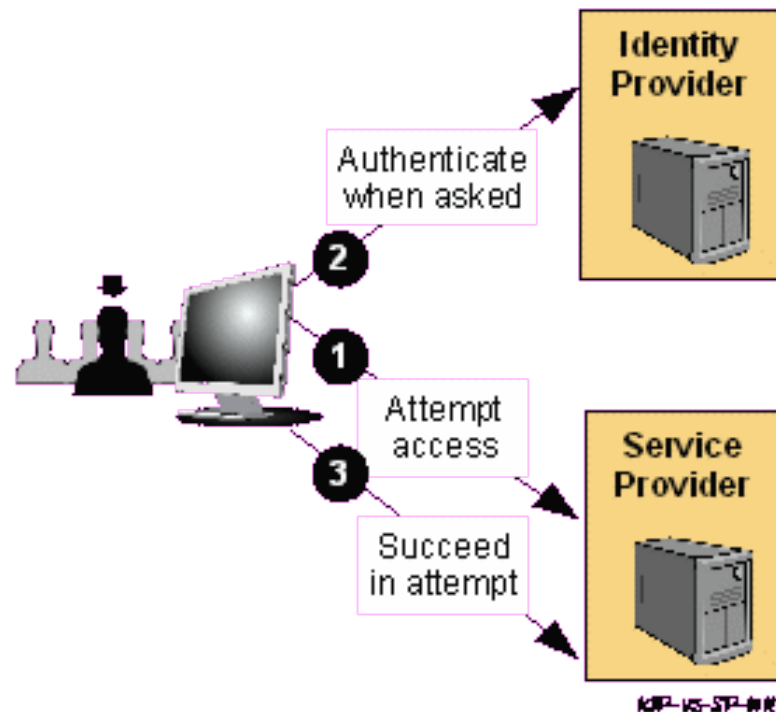
SAML request
or
SAML response

protocol SOAP HTTP

Initiation of web browser SSO

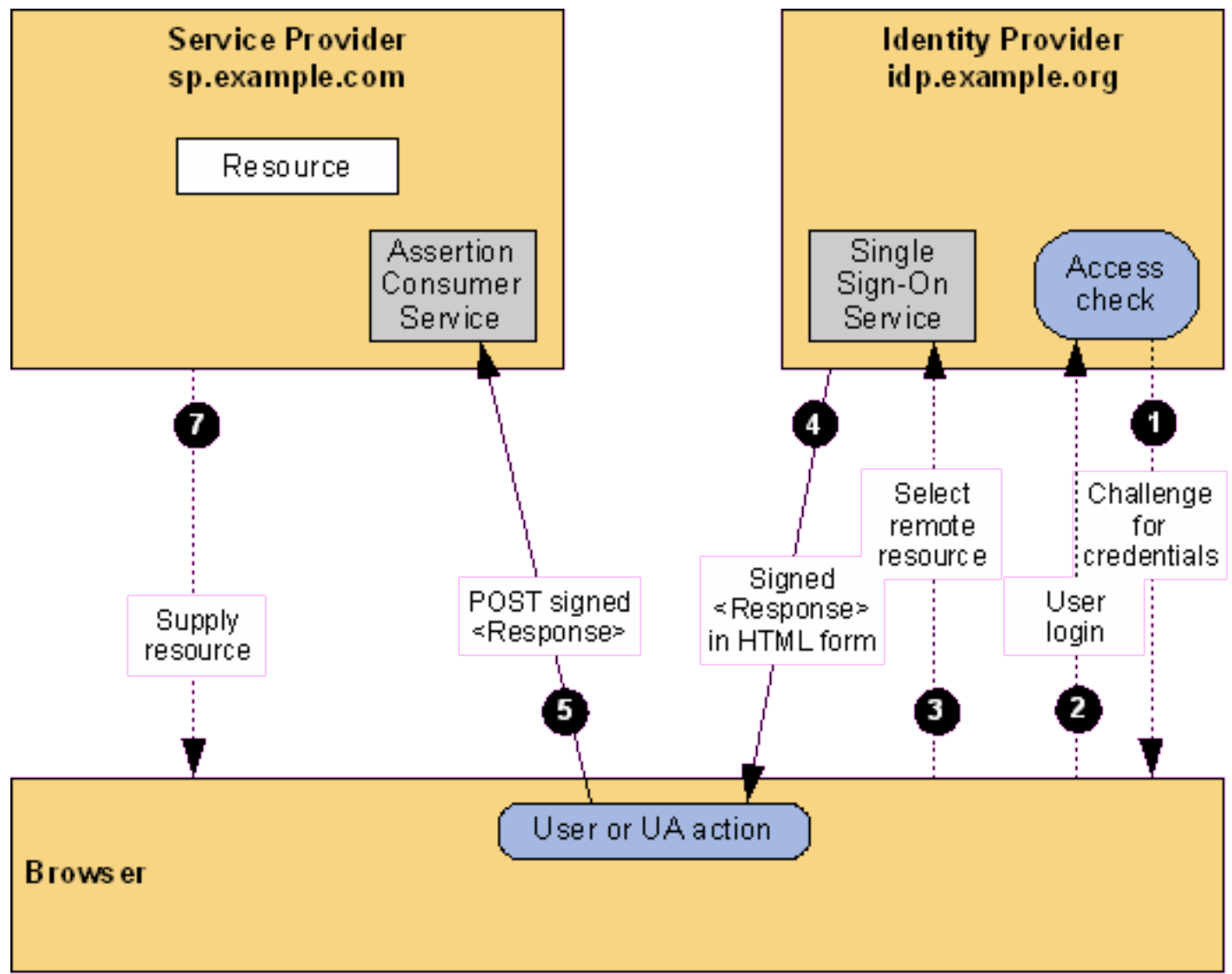


IDP-initiated



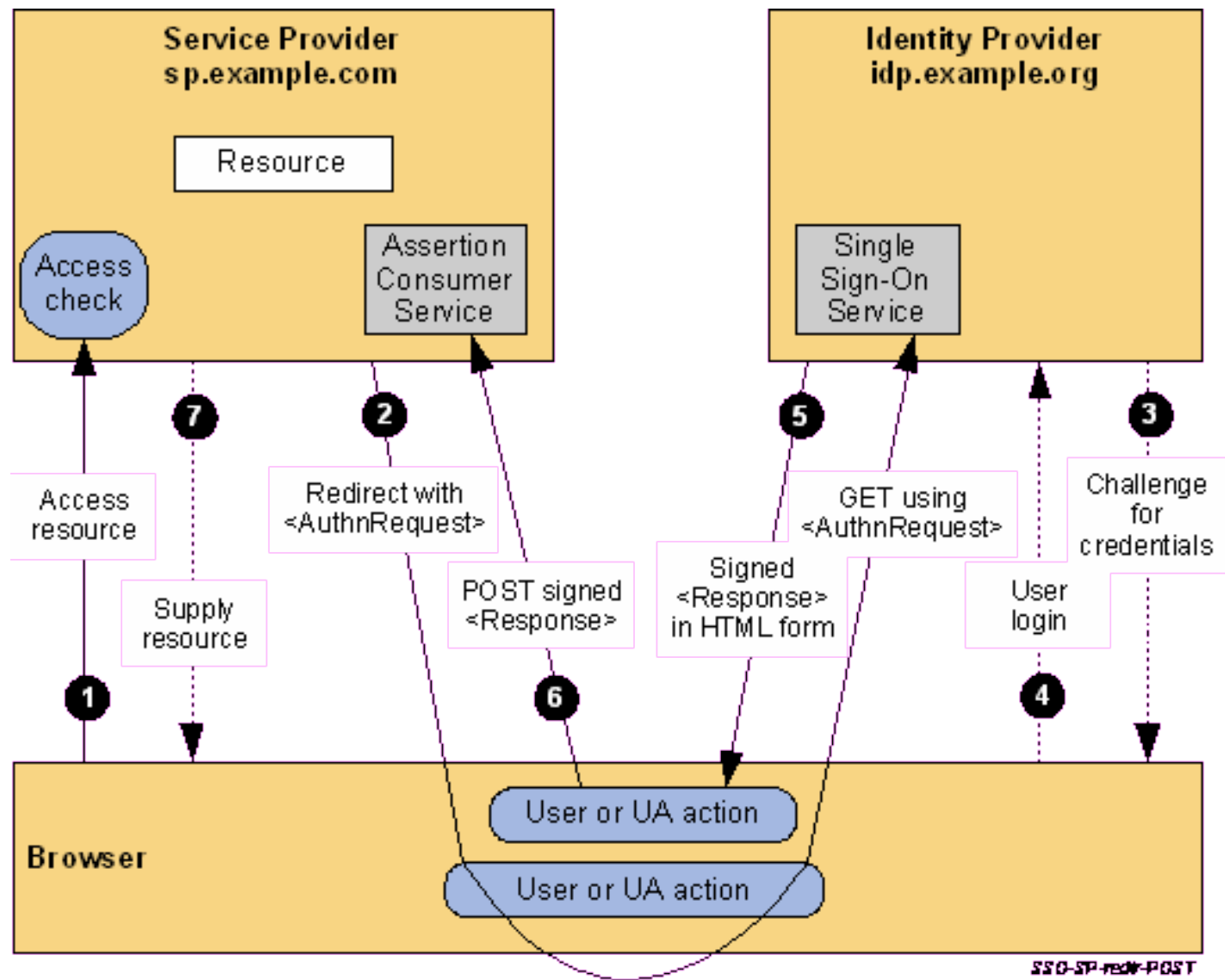
SP-initiated

Example of IdP-Initiated SSO: POST Bindings

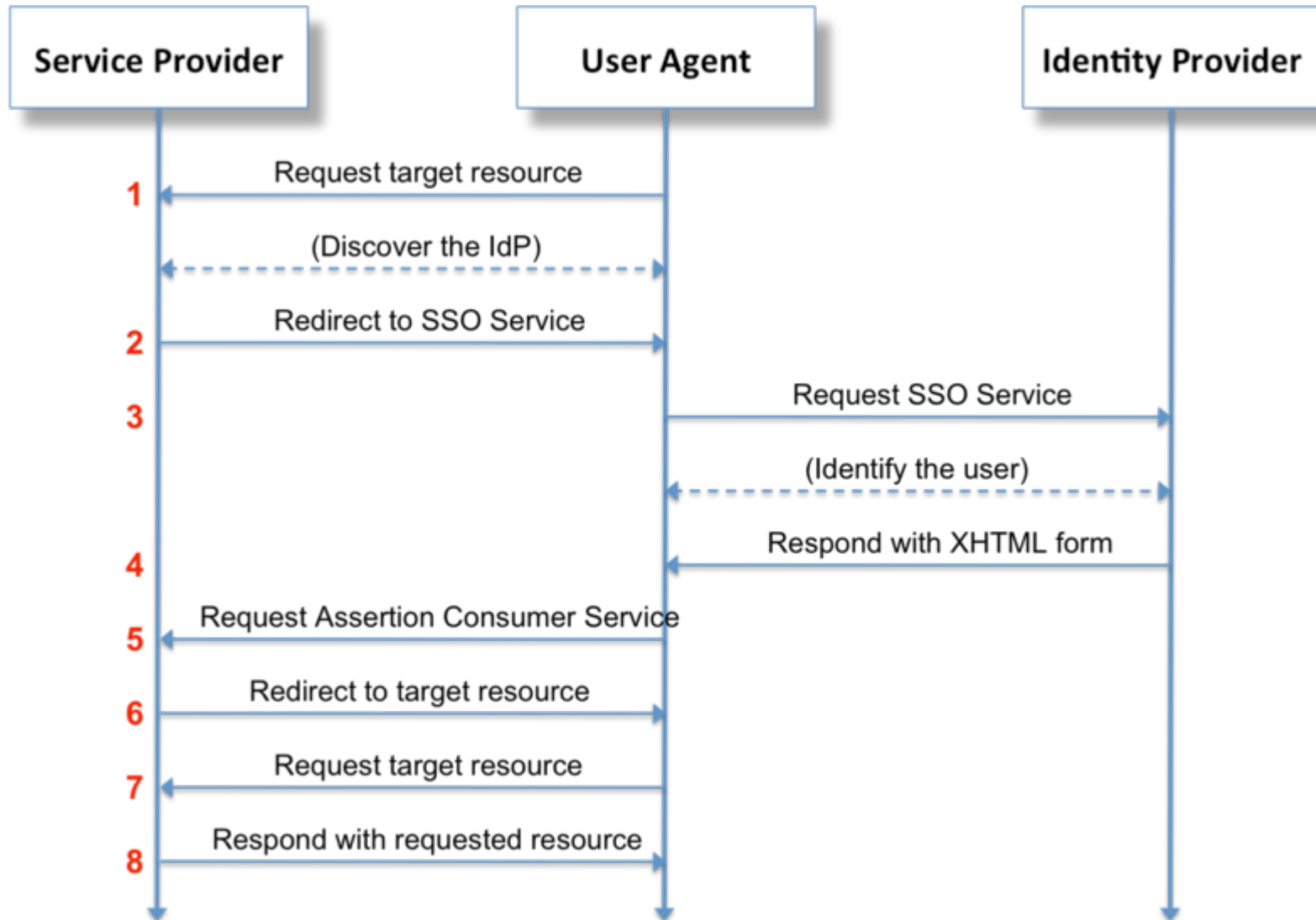


SSO-POST

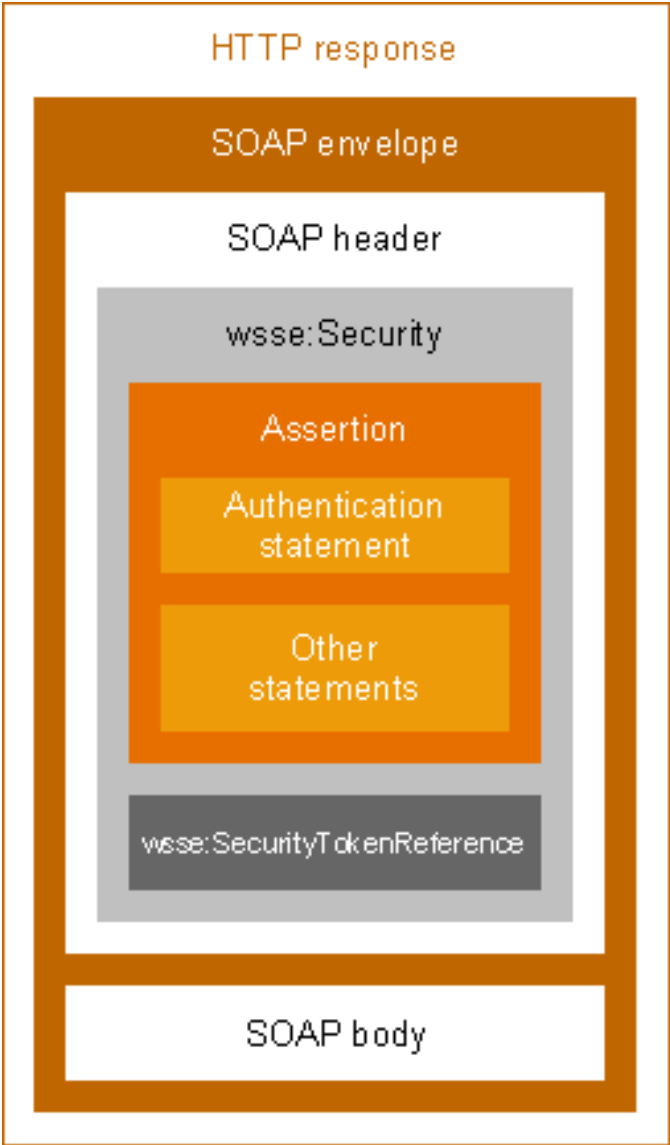
Example of SP-Initiated SSO: Redirect/POST Bindings



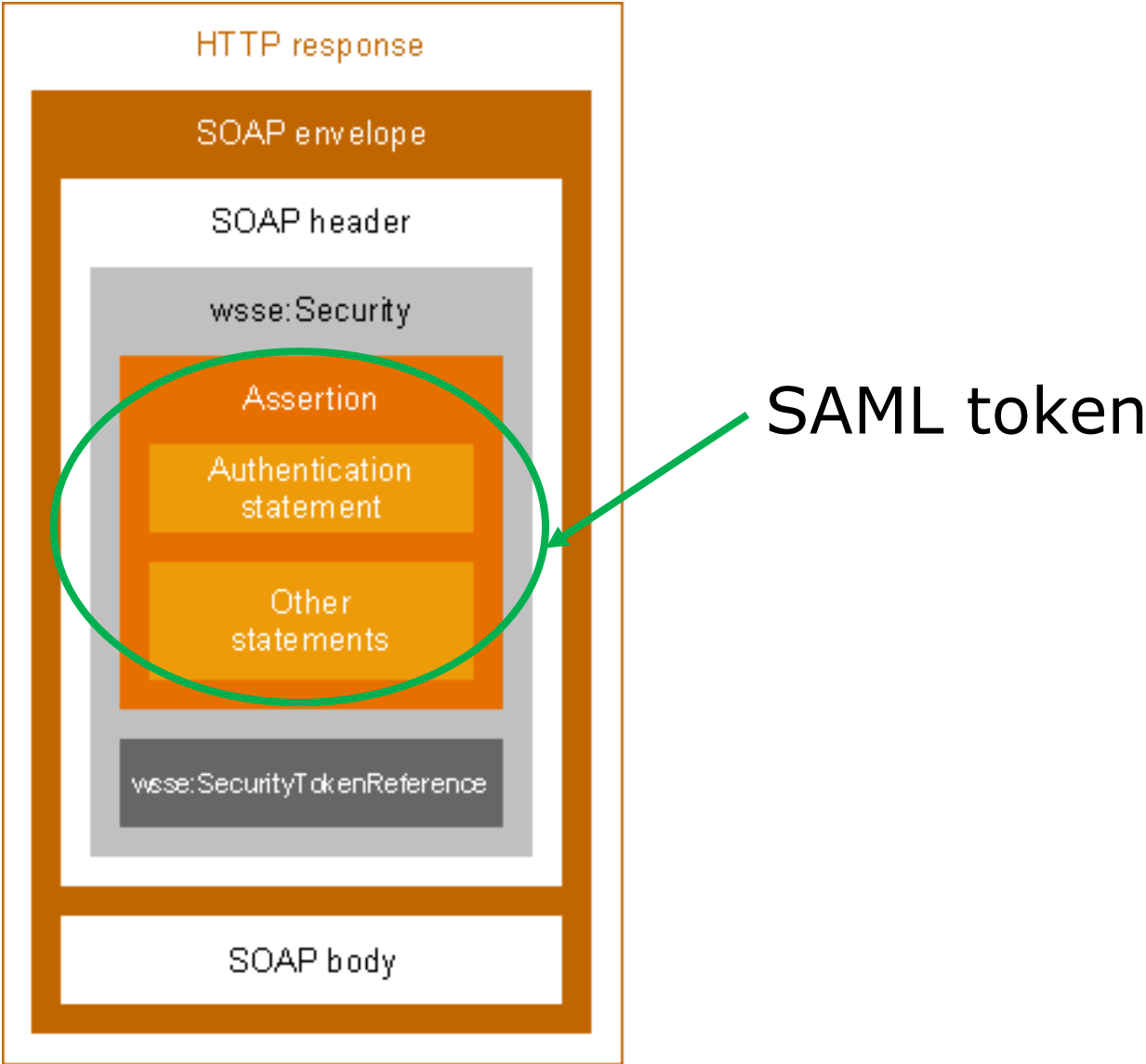
SP-Initiated web browser SSO example



SAML extensions. Example: Web Services Security (WS-Security)



SAML extensions. Example: Web Services Security (WS-Security)



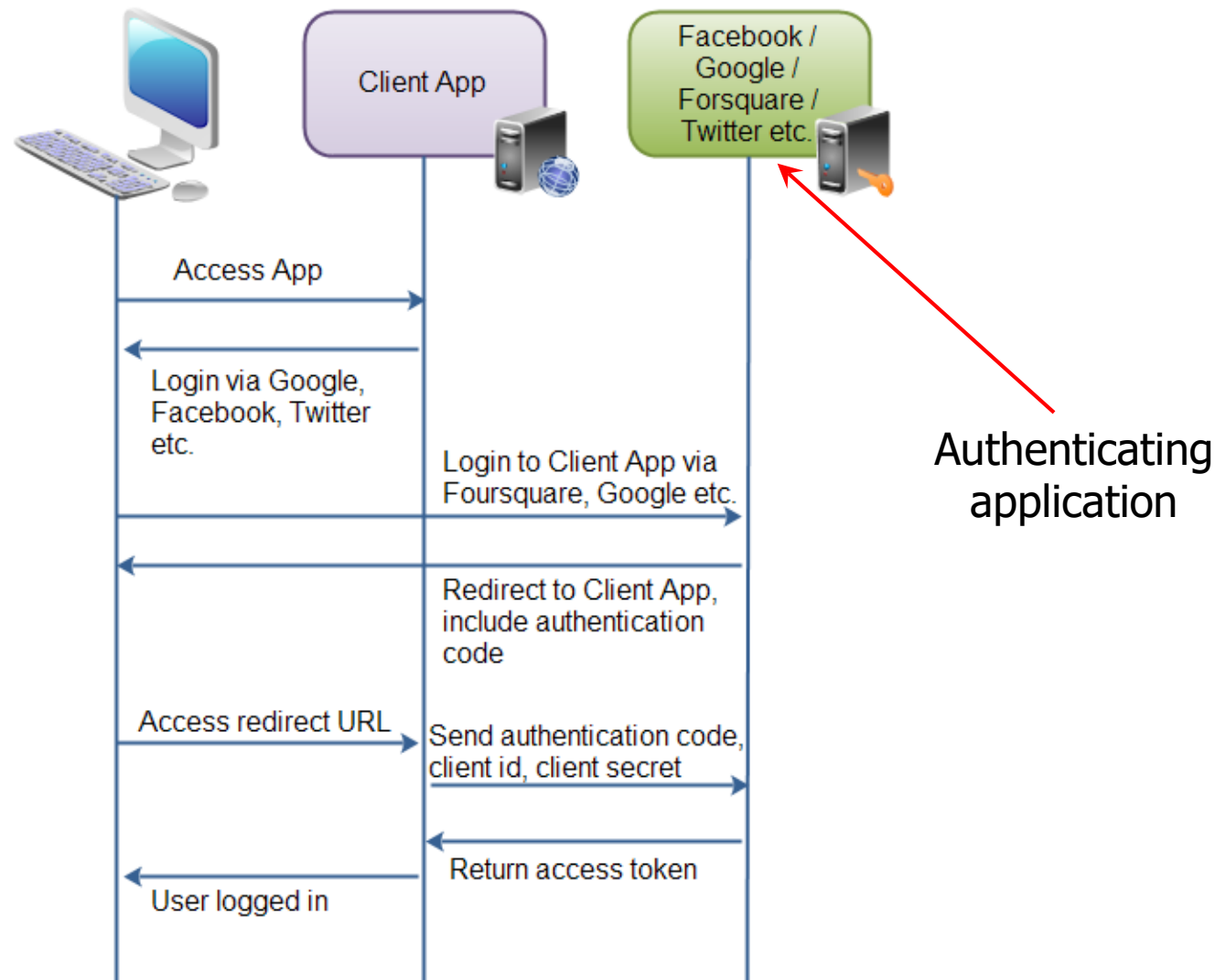
OAuth 2.0: Overview

- Developed by IETF
- Published in RFC 6749 in 2012
 - Not backwards compatible with OAuth 1.0 !!
- OAuth 1.0 was published in RFC 5849 in 2010
 - Implements different authorization flows
 - No longer used in many services (e.g. Google APIs)
 - <https://developers.facebook.com/docs/instagram-basic-display-api/getting-started>
 - <https://developers.facebook.com/docs/instagram-platform>

OAuth 2.0: Features

- Authentication protocol
- Allows users to approve an application to act on their behalf
- No password share with the application
- Text protocol
- Over HTTP(S)

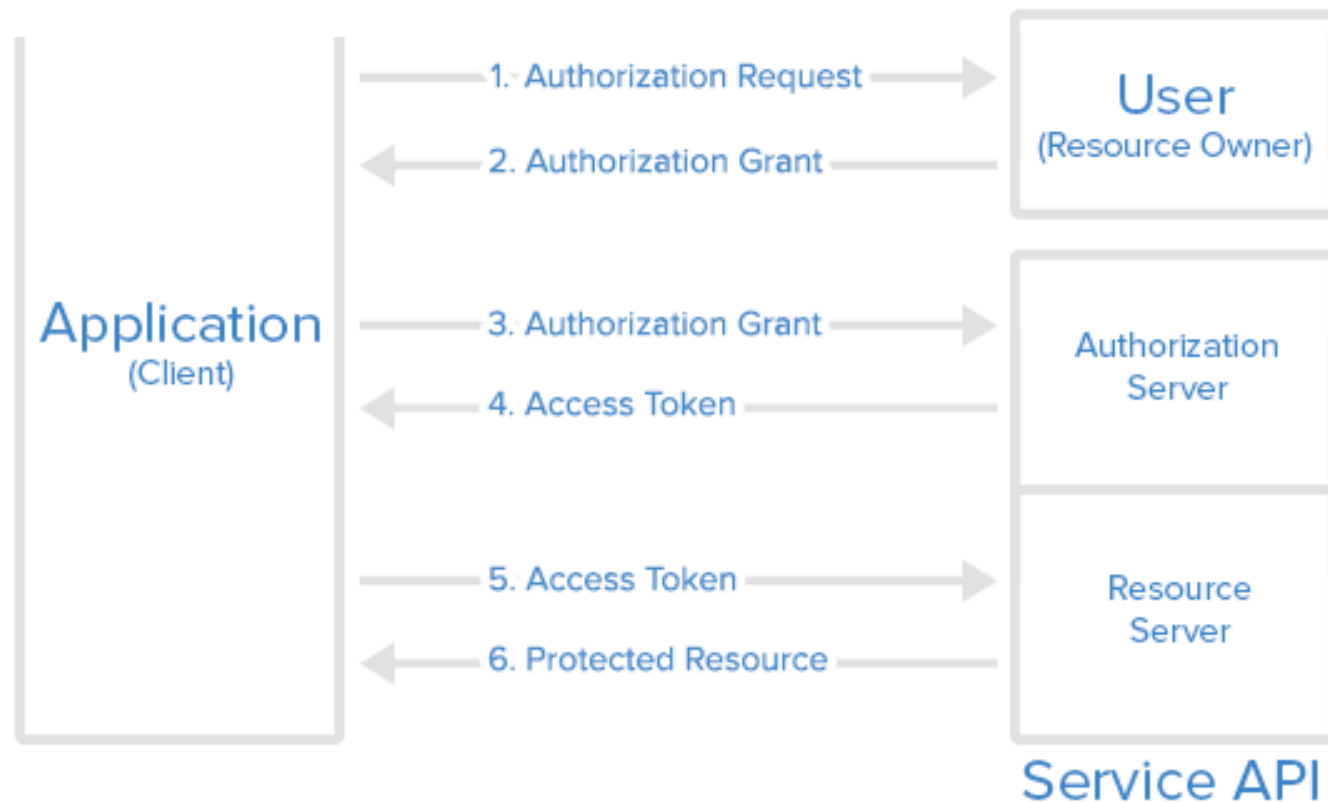
OAuth 2.0: Protocol example



Source: <http://tutorials.jenkov.com/oauth2/overview.html>

OAuth 2.0: Abstract Protocol Flow

Abstract Protocol Flow



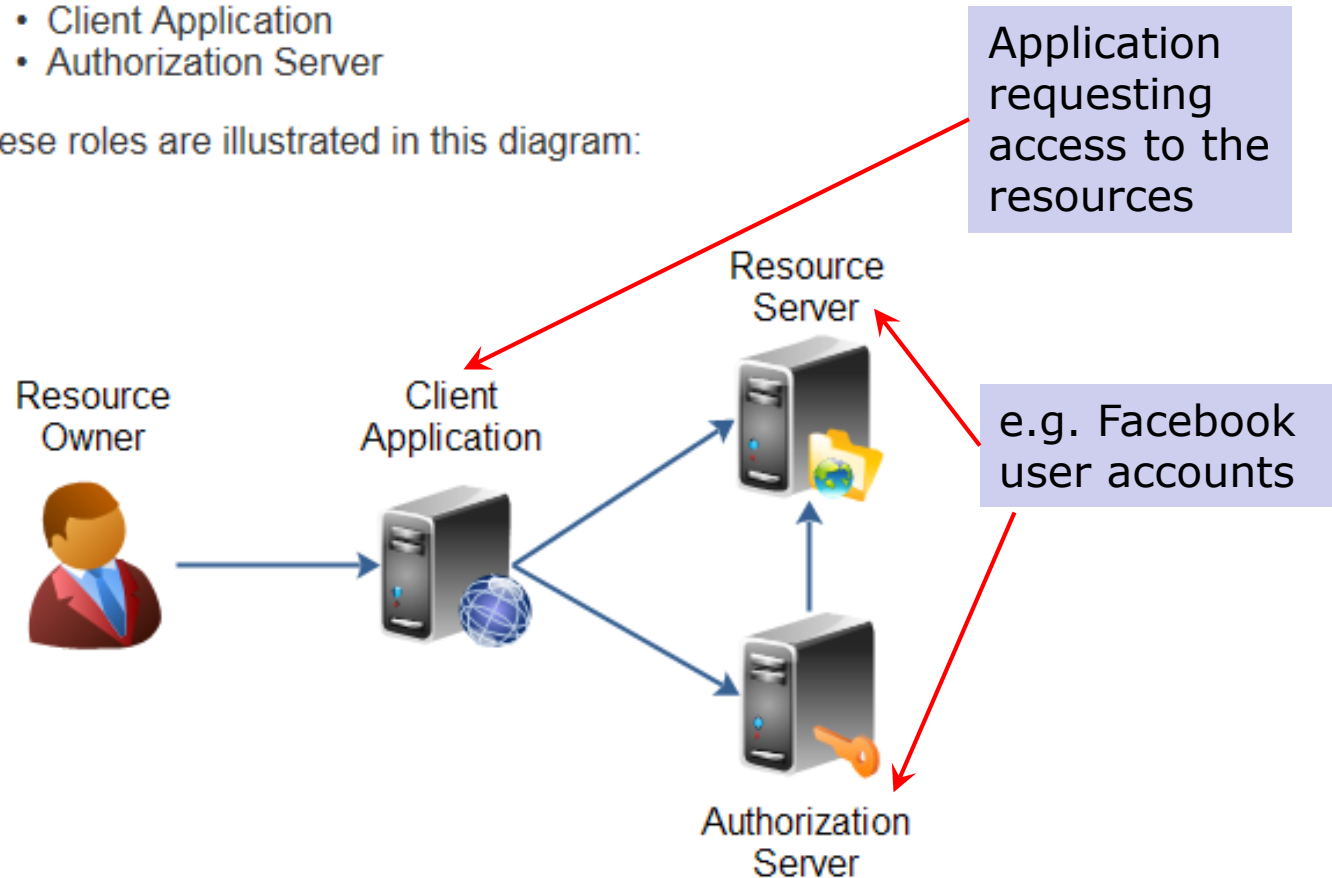
Source: <https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2>

OAuth 2.0: Roles

OAuth 2.0 defines the following roles of users and applications:

- Resource Owner
- Resource Server
- Client Application
- Authorization Server

These roles are illustrated in this diagram:

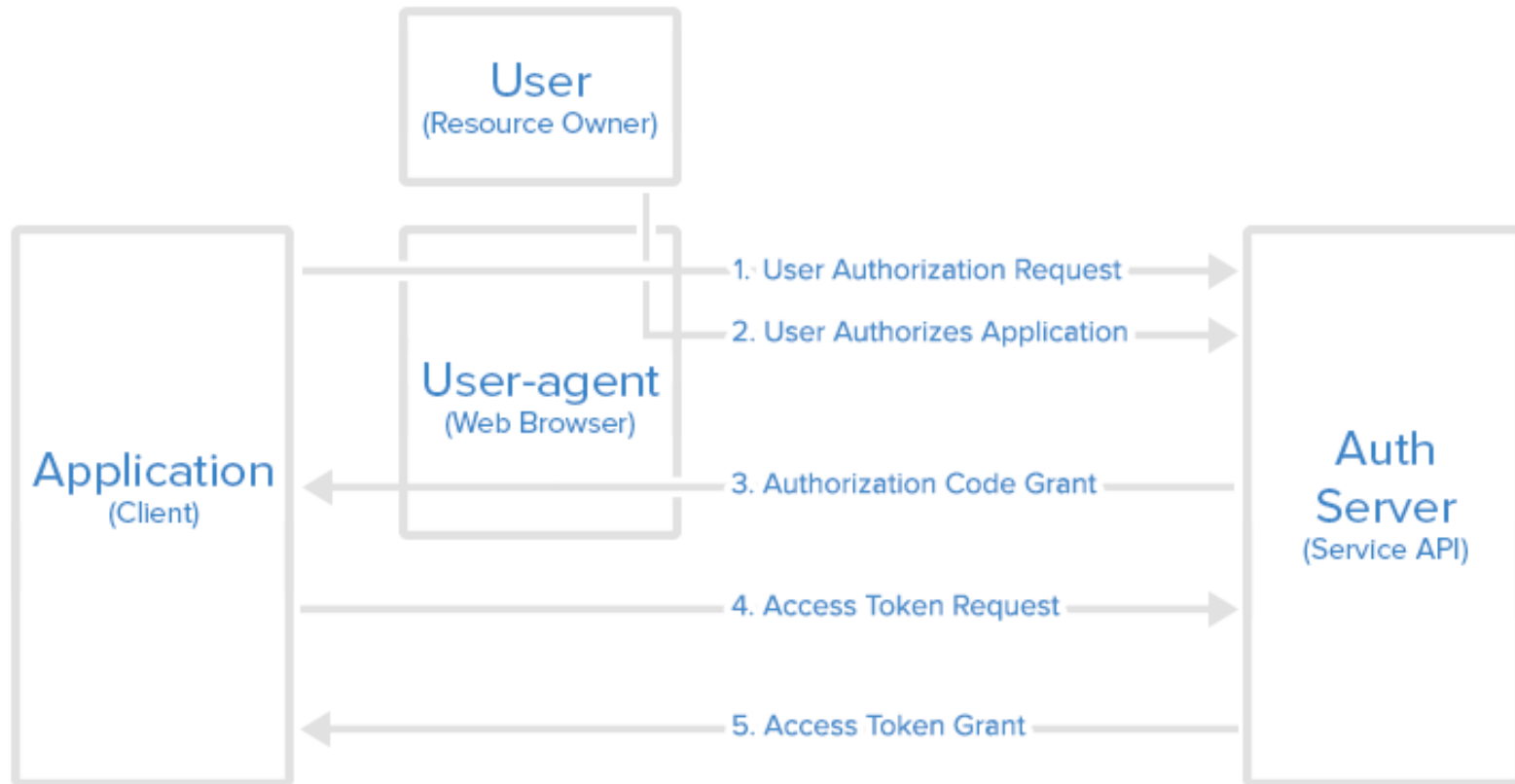


OAuth 2.0 roles as defined in the specification.

Authorization Grant types

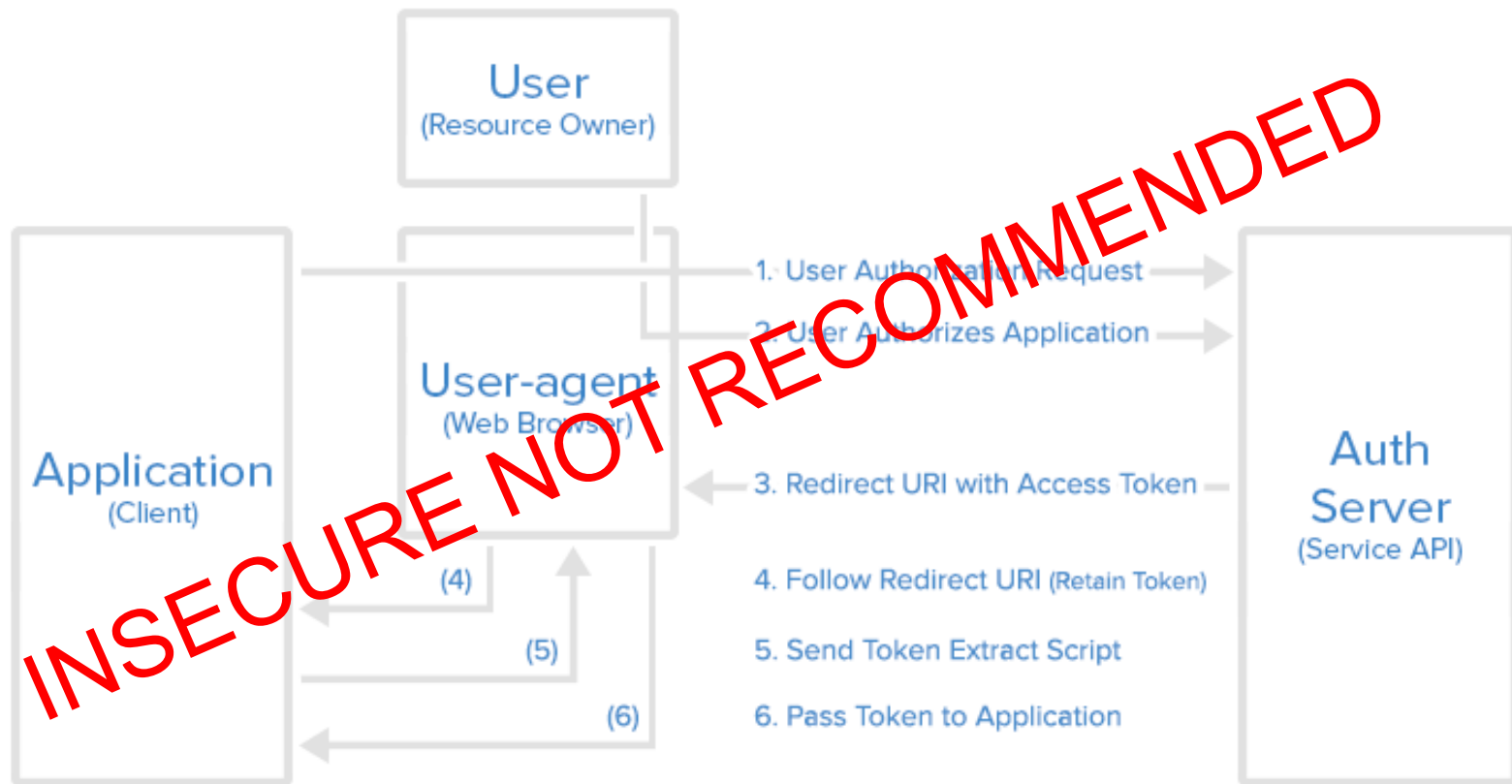
- Authorization code:
 - Server-side applications (*Most commonly used*)
- Implicit:
 - Applications running on user's device
- Resource Owner password credentials:
 - Trusted applications (owned by the service, ...).
- Client credentials:
 - Applications API access

Grant Type: Authorization Code Flow



Source: <https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2>

Grant Type: Implicit flow



Source: <https://www.digitalocean.com/community/tutorials/an-introduction-to-oauth-2>

OAuth 2.0: Authorization request

response_type

Required. Must be set to code

client_id

Required. The client identifier as assigned by the authorization server, when the client was registered

redirect_uri

Optional. The redirect URI registered by the client

scope

Optional. The possible scope of the request

state

Optional (recommended). Any client state that needs to be passed on to the client request URI

OAuth 2.0: Authorization response

code

Required. The authorization code

state

Required, if present in request. The same value as sent by the client in the state parameter, if any

OAuth 2.0: Token Request

`client_id`

Required. The client application's id

`client_secret`

Required. The client application's client secret

`grant_type`

Required. Must be set to `authorization_code`

`code`

Required. The authorization code received by the authorization server

`redirect_uri`

Required, if the request URI was included in the authorization request. Must be identical then

OAuth 2.0: Token Response

```
{ "access_token" : "...",  
  "token_type" : "...",  
  "expires_in" : "...",  
  "refresh_token" : "...",  
}
```

JSON array

Properties:

- `access_token` as assigned by the authorization server
- `token_type` assigned by the authorization server
- `expires_in` is a number of seconds after which the access token expires. *Expiration of access tokens is optional*
- `refresh_token` in case the access token can expire. It is used to obtain a new access token

OAuth 2.0: Web Server Applications

- Token Request: Google APIs example
- REST request example:

```
POST /oauth2/v3/token HTTP/1.1
```

```
Host: www.googleapis.com
```

```
Content-Type: application/x-www-form-urlencoded
```

```
code=4/P7q7W91a-oMsCeLvIaQm6bTrgtp7&
```

```
client_id=8819981768.apps.googleusercontent.com&
```

```
client_secret={client_secret}&
```

```
redirect_uri=https://oauth2-login-demo.appspot.com/code&
```

```
grant_type=authorization_code
```

Source: <https://developers.google.com/identity/protocols/OAuth2>

OAuth 2.0: Web Server Applications

- Token Response: Google APIs example
- JSON response example (in the HTTP response):

```
{  
  "access_token": "1/fFAGRNJru1FTz70BzhT3Zg",  
  "expires_in": 3920,  
  "token_type": "Bearer"  
}
```

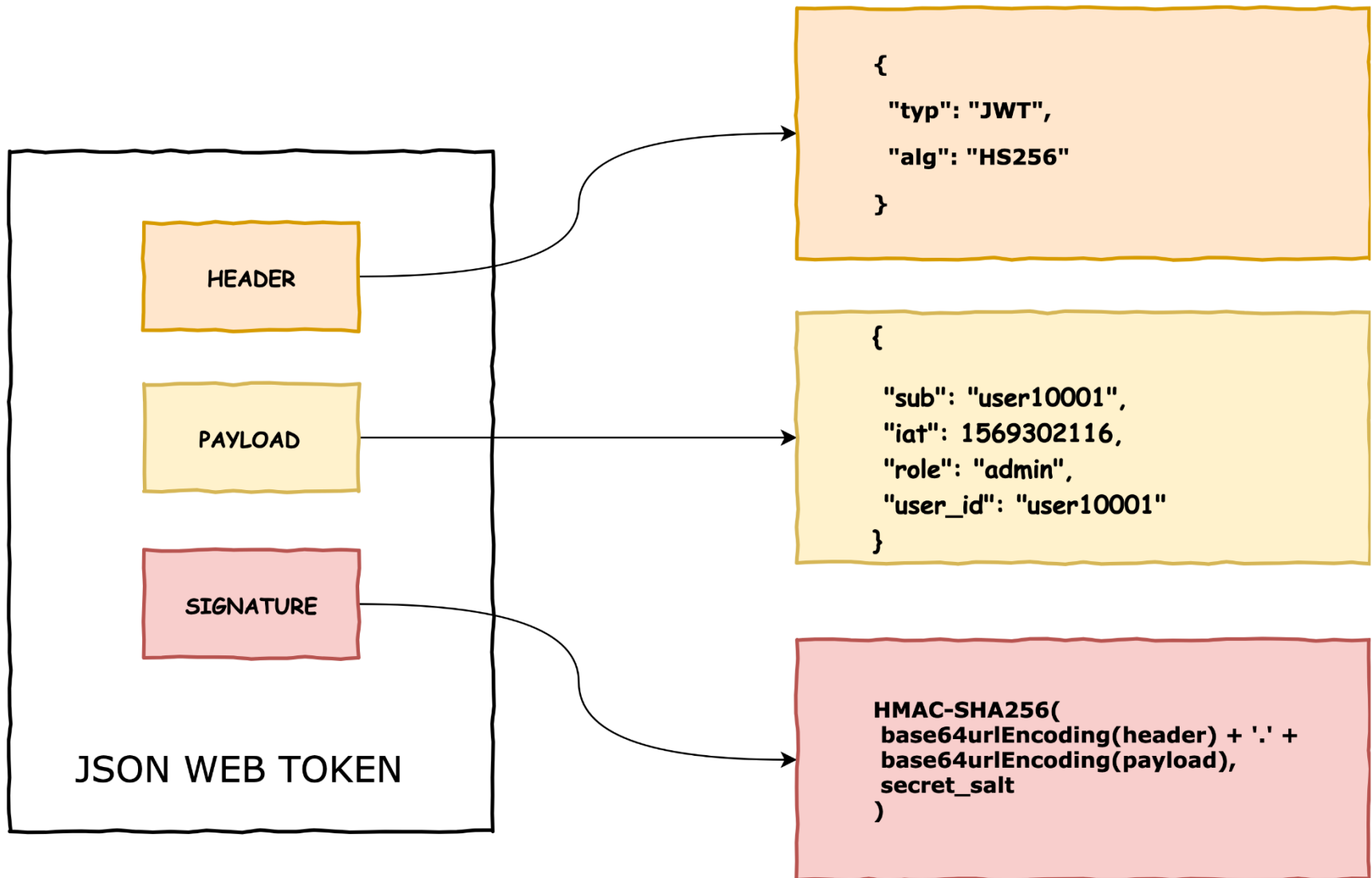
Source: <https://developers.google.com/identity/protocols/OAuth2>

JSON Web Tokens (JWT)

- SAML simplification
- “JWT (“jot”) is a standard for safely passing claims in space constrained environments” (*)
 - JSON Web Token (JWT) RFC7519
 - JSON Web Signature (JWS) RFC7515
 - JSON Web Encryption (JWE) RFC 7516
- Defines a container format for:
 - User authentication, secure information exchange

* References: * The JWT Handbook, S.E. Peyrott
<https://datatracker.ietf.org/doc/html/rfc7519>
<https://datatracker.ietf.org/doc/html/rfc7515>
<https://datatracker.ietf.org/doc/html/rfc7516>

JWT structure



Source: <https://sureshnsk.dev/how-json-web-token-jwt-authentication-works>

JSON Web Tokens - jwt.io

Ryan

← → ↺ 🏠

jwt.io

🇺🇸 ☆ ☰

 JWT

Debugger Libraries Ask Get a T-shirt!

ALGORITHM

HS256 ▼

Encoded

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoiNjUyMzQ0MDAwMC4yJVA95OrM7E2cBab30RMHrHDCEfxbjoYZgeFONFh7HgQ
```

Decoded

HEADER:

```
{  "alg": "HS256",  "typ": "JWT"}
```

PAYLOAD:

```
{  "sub": "1234567890",  "name": "John Doe",  "admin": true}
```

VERIFY SIGNATURE

HMACSHA256(
base64UrlEncode(header) + "." +
base64UrlEncode(payload),

secret

)

☐ secret base64 encoded

✔ Signature Verified



UPC DMAG
DISTRIBUTED MULTIMEDIA APPLICATIONS GROUP

JWT structure

3 parts separated by '.'

Header

Payload

Signature (optional)

JSON Web Tokens - jwt.io

JWT

Debugger Libraries Ask Get a T-shirt!

ALGORITHM HS256

Encoded

Decoded

HEADER:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD:

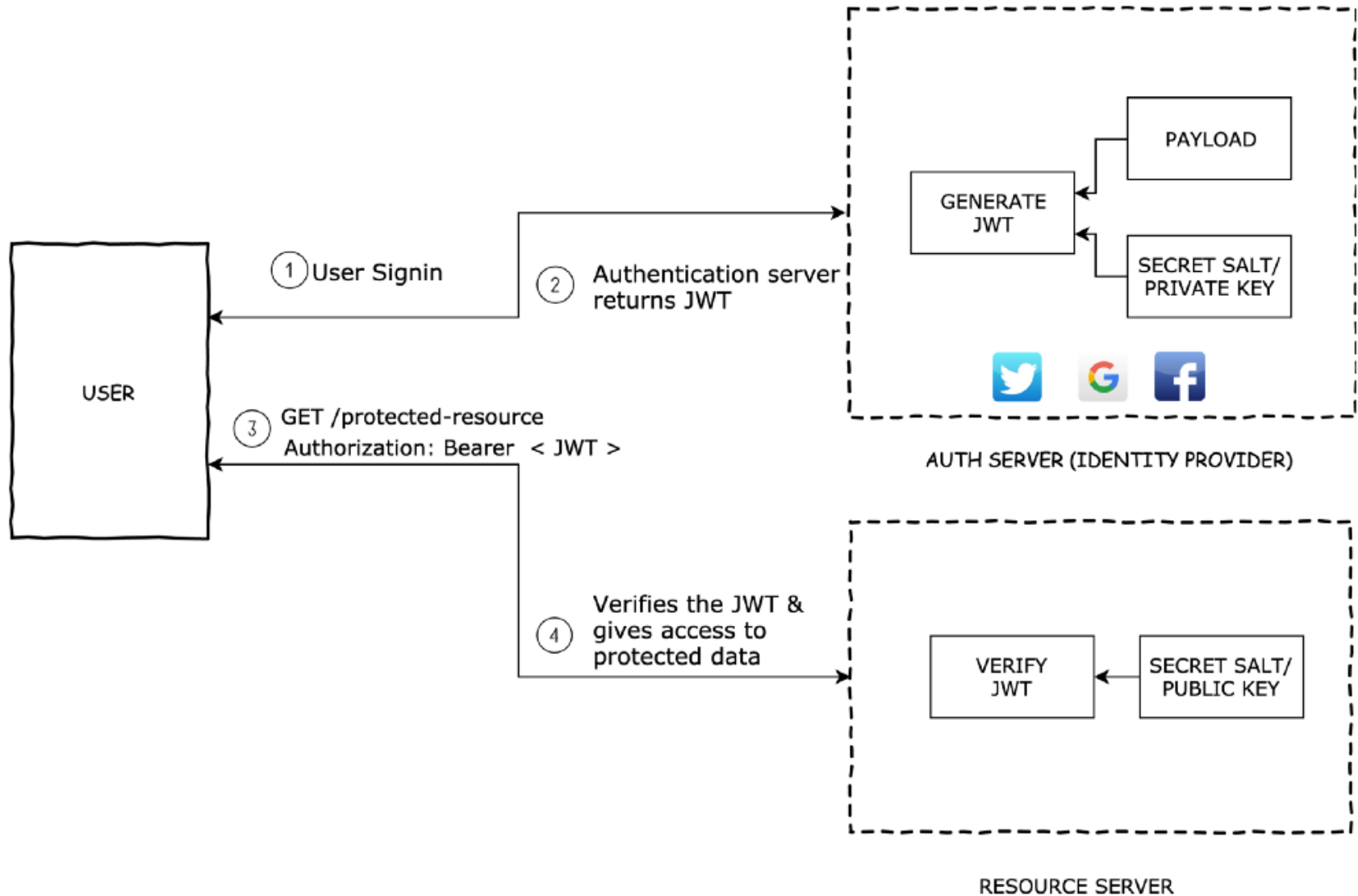
```
{
  "sub": "1234567890",
  "name": "John Doe",
  "admin": true
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  secret
) ☐ secret base64 encoded
```

Signature Verified

JWT example of use



Security in applications

- **1.** Protocols:
 - Web (HTTP)
 - E-mail (content)
- **2.** Formats:
 - XML: XML Encryption, XML Signature
- **3.** Protocols specific for security: SAML, OAuth, JWT
- Others:
 - **4. Data protection / Privacy / Digital identity**
 - **5.** Intellectual rights for multimedia content

Curiosity: Google Account 1

Cómo se personalizan tus anuncios

Los anuncios se basan en la información personal que has añadido a tu cuenta de Google, en los datos de los anunciantes asociados a Google y en los intereses de usuario que determina Google. Elige cualquier factor para obtener más información al respecto o modificar tus preferencias. [Más información](#)



35-64 años



Hombre



Diseño y desarrollo web



Estado civil: Personas casadas



Estado de propiedad de la vivienda: En propiedad



Estado parental: Sin hijos



Informática y electrónica



Nivel de estudios: Graduado en educación secunda...



Redes



Sector laboral: Sector tecnológico



Tamaño de la empresa: Pequeños empresarios (entr...



Televisión y vídeo

Curiosity: Google Account 1 (next day)

 18-34 años	 Mujer
 Decoración del hogar y de interiores	 Diseño y desarrollo web
 Equipo de cámara y fotografía	 Estado civil: En una relación
 Estado de propiedad de la vivienda: En propiedad	 Estado parental: Sin hijos
 Facultades y universidades	 Fotografía y arte digital
 Informática y electrónica	 Jubilación y pensiones
 Mobiliario doméstico	 Nivel de estudios: Estudiantes universitarios (en la a...
 Películas	 Redes
 Sector laboral: Sector tecnológico	 Teléfonos móviles
 Televisión y vídeo	

Curiosity: Google Account 2

Cómo se personalizan tus anuncios

Los anuncios se basan en la información personal que has añadido a tu cuenta de Google, en los datos de los anunciantes asociados a Google y en los intereses de usuario que determina Google. Elige cualquier factor para obtener más información al respecto o modificar tus preferencias. [Más información](#)



35-64 años



Estado civil: Personas casadas



Estado de propiedad de la vivienda: En propiedad



Estado parental: Sin hijos



Inversión



Nivel de estudios: Grado universitario



Noticias



Noticias financieras



Sector laboral: Sector financiero



Servicios para empresas



Tamaño de la empresa: Grandes empresarios (entr...

Curiosity: Google Account 2 (next day)

 35-64 años	 Actividades al aire libre
 Alimentos para gourmets y especialidades	 Animales de compañía
 Automóviles y vehículos	 Ciclismo
 Ciencia ficción y fantasía en televisión	 Cine de acción y aventura
 Cine de animación	 Cocina y recetas
 Correr y pasear	 Decoración del hogar y de interiores
 Deportes	 Domótica
 Estado civil: Personas casadas	 Estado de propiedad de la vivienda: En propiedad
 Estado parental: Sin hijos	 Familia
 Fitness	 Fútbol americano
 Hardware informático	 Informática distribuida y cloud computing
 Informática y electrónica	 Inversión
 Juegos de ordenador y videojuegos	 Libros y literatura
 Mejora del hogar	 Mudanzas y traslados
 Nivel de estudios: Grado universitario	 Noticias
 Noticias financieras	 Películas
 Películas de ciencia ficción y fantasía	 Redes sociales
 Restaurantes	 Ropa
 Sector laboral: Sector tecnológico	 Servicios para empresas

- Personally Identifiable Information (PII) in digital form
 - Social Networks
 - Public institutions (i.e. health organizations, etc.)
 - ...
- How to control access to this information?
 - Privacy policies/rules. XACML
 - Privacy controls (organizational, physical and technical measures) to treat risks
 - Enforcement. Protection. Access control

- Policies
- Expression: XACML, ...
- Enforcement

- Digital identity management

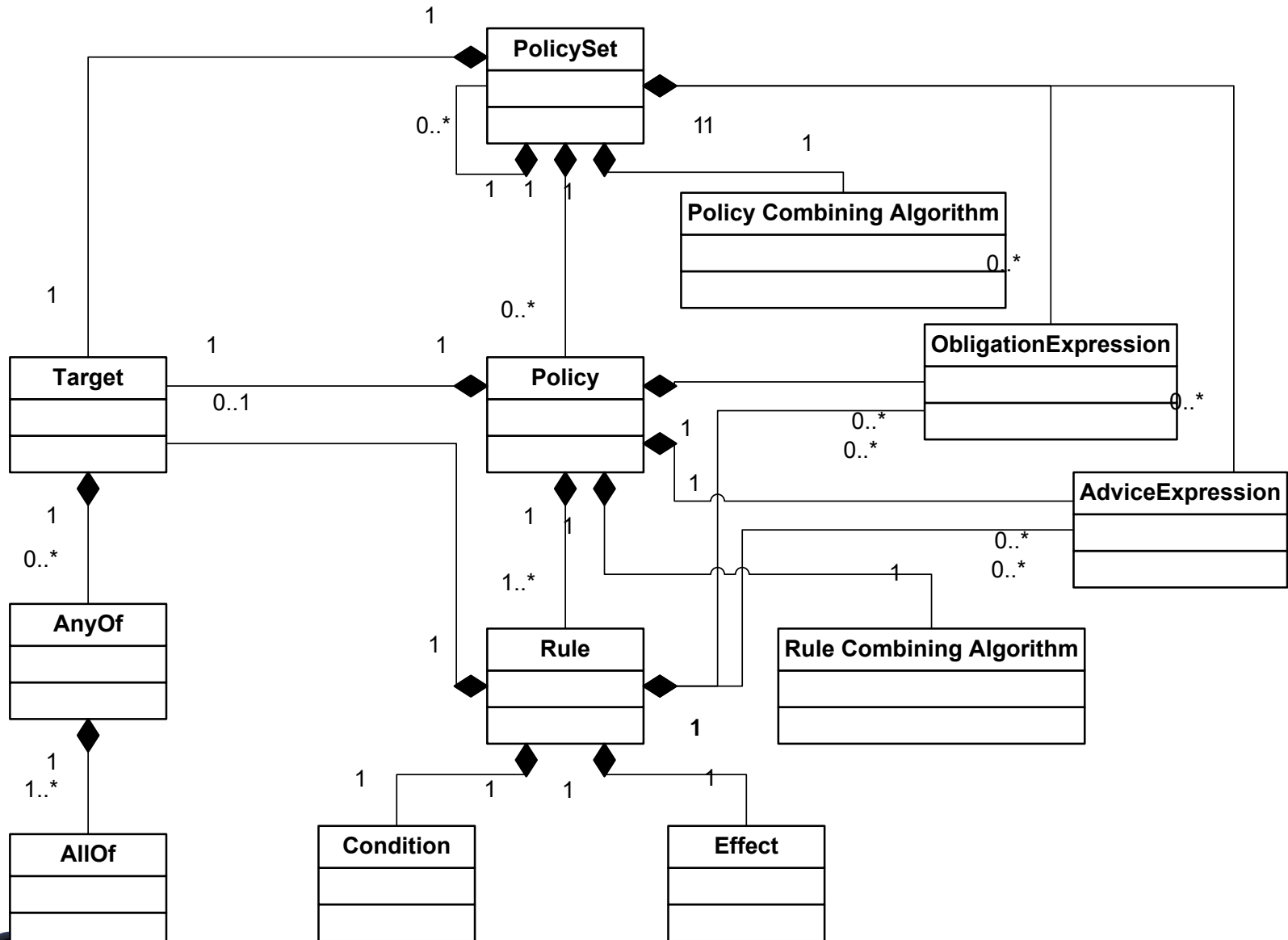
- **XACML (eXtensible Access Control Markup Language)**
- OASIS Standard. Version 3.0, 22nd January 2013, revised 2017. Version 2.0 in 2005.

<http://docs.oasis-open.org/xacml/3.0/xacml-3.0-core-spec-os-en.pdf>

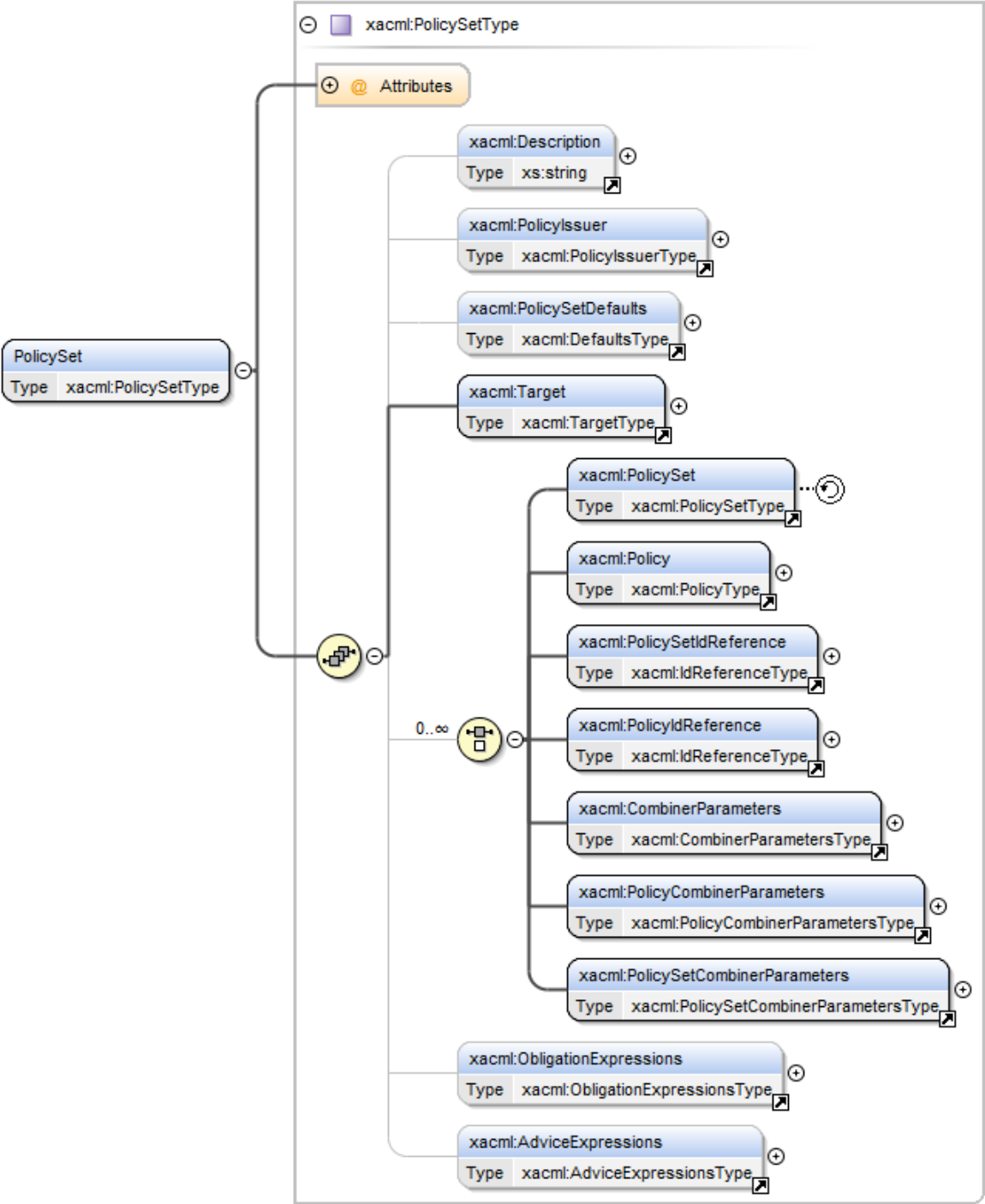
<http://docs.oasis-open.org/xacml/3.0/errata01/os/xacml-3.0-core-spec-errata01-os.pdf>

- Designed to express authorization policies in XML over objects identifiable in XML
- Specifies a model for policies formed by 3 elements:
 - Rule
 - Policy
 - PolicySet

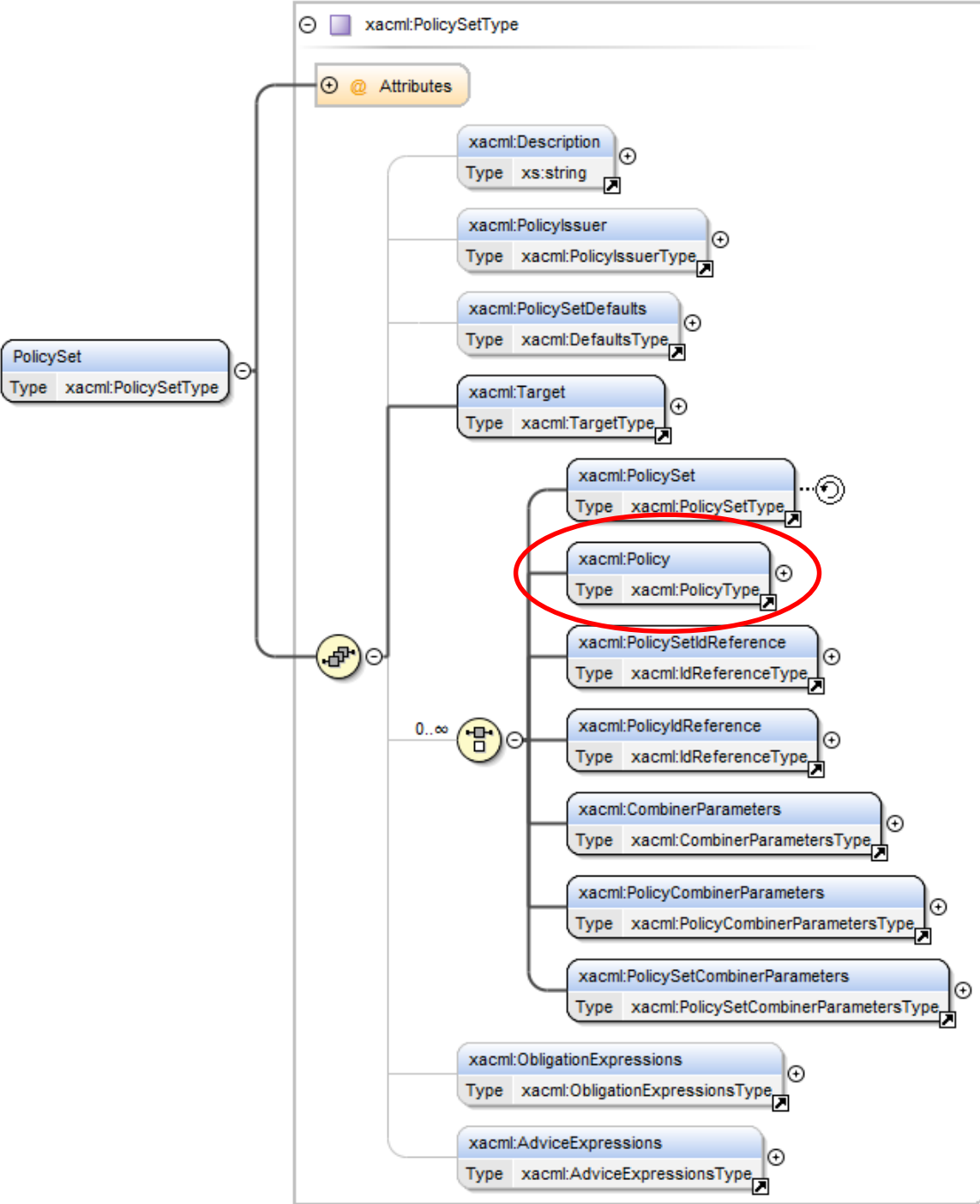
XACML 3.0 Policy Model



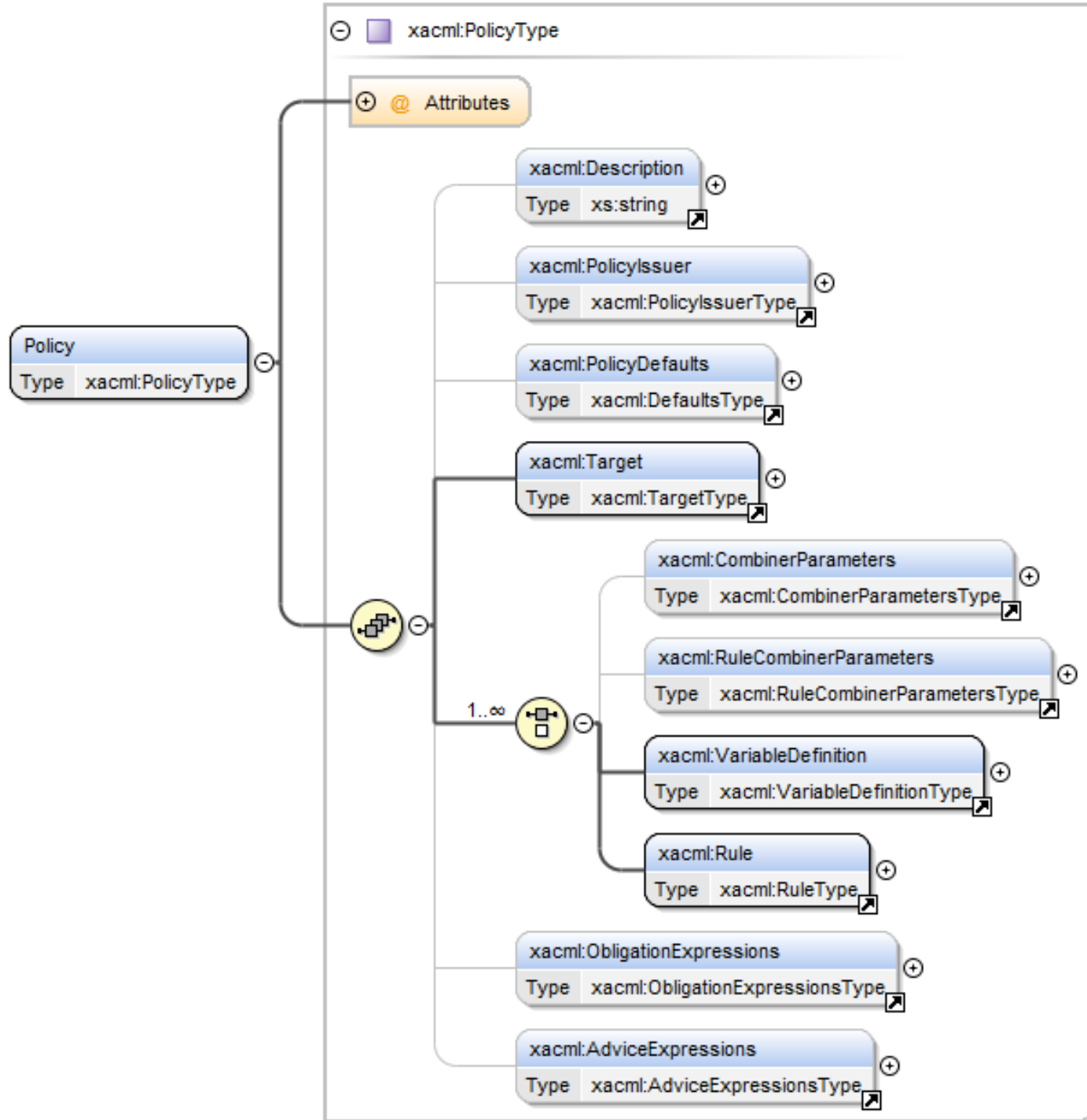
PolicySet Element



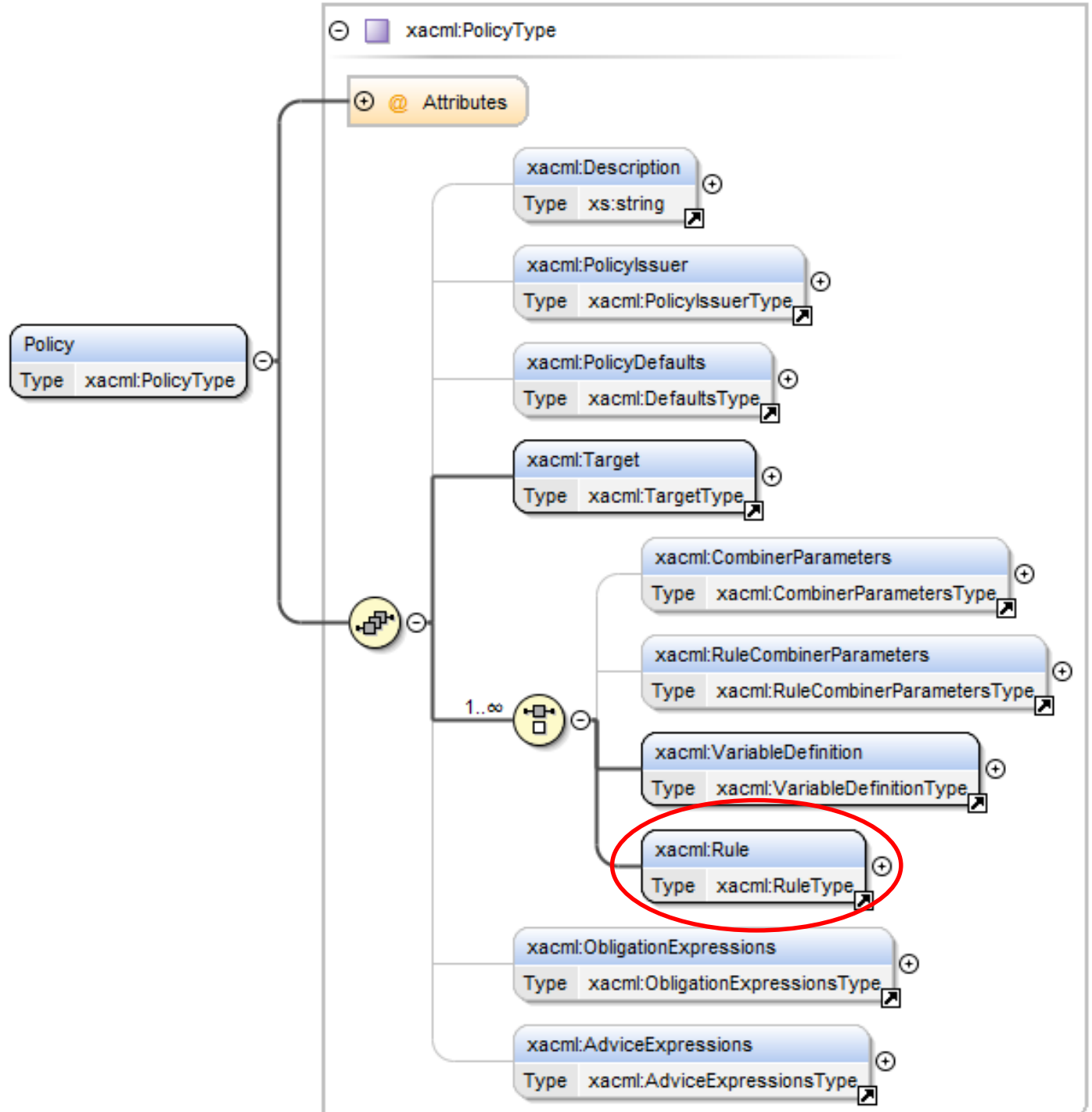
PolicySet Element



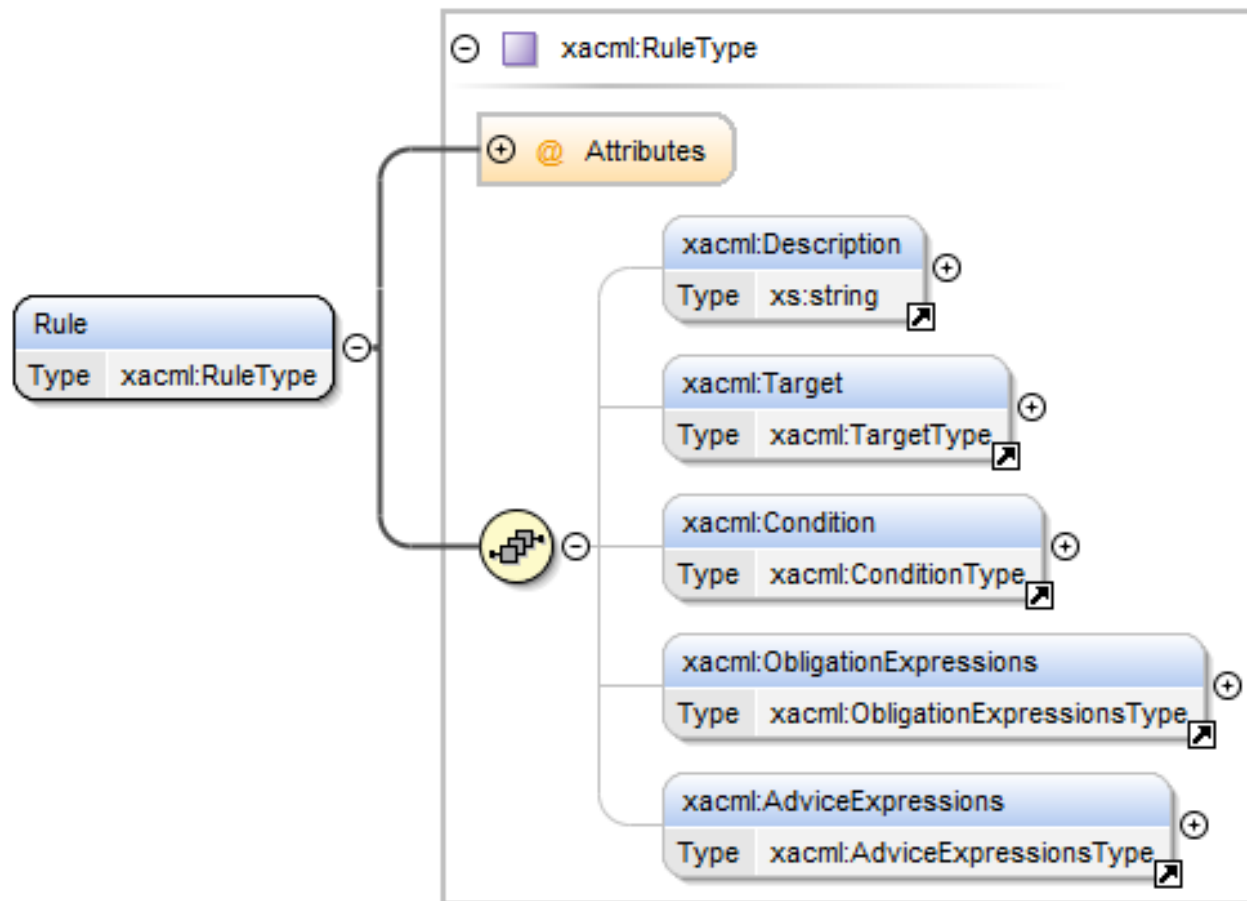
Policy Eleme



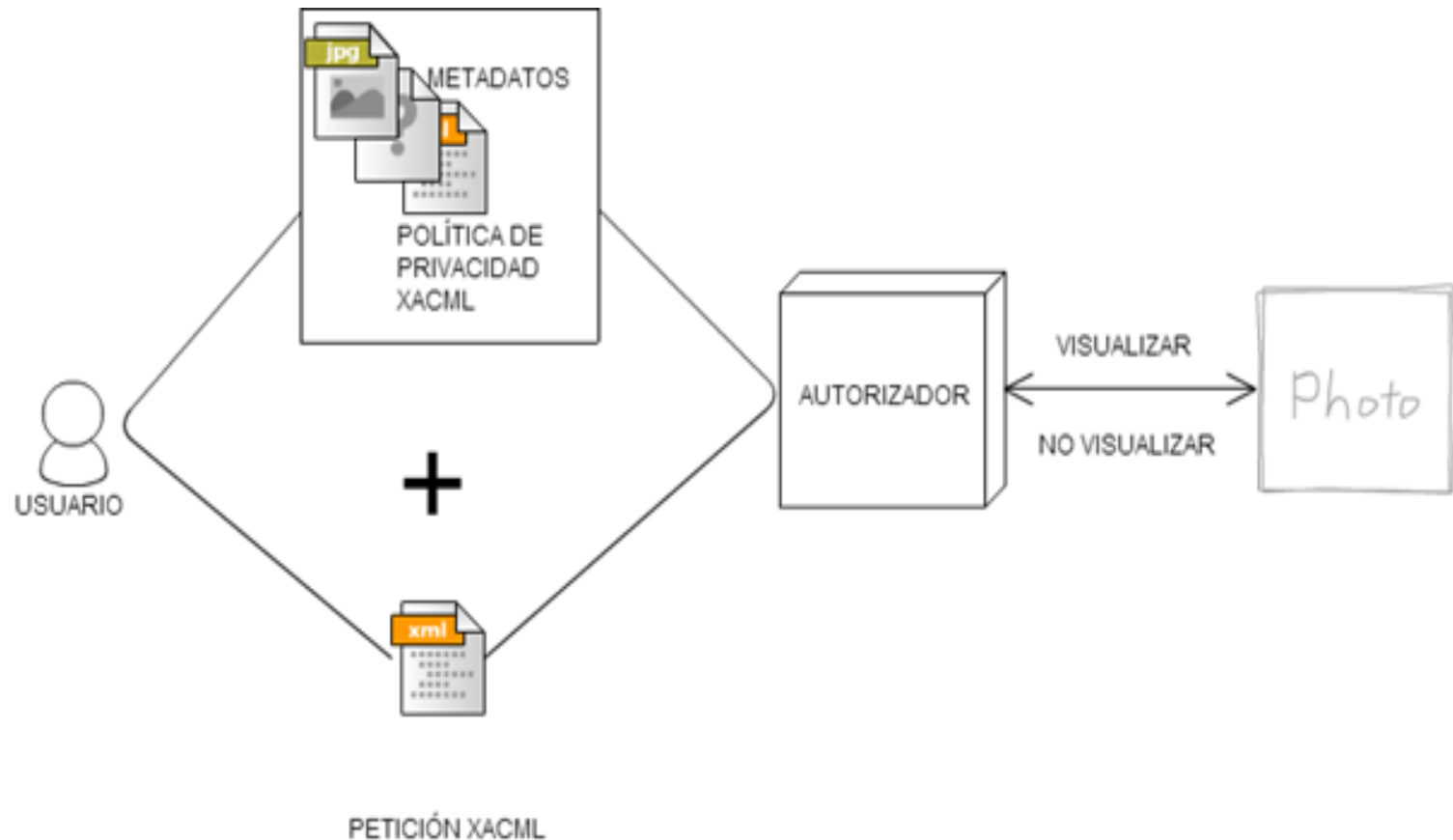
Policy Eleme



Rule element



Example of use of XACML



Source:

<https://upcommons.upc.edu/handle/2117/82555>

<https://upcommons.upc.edu/handle/2117/117807>

Example: Privacy in eHealth

- Digital medical information
 - Lab tests, radiology results, discharge reports, etc.
- Management of medical documents
 - **Security** (confidentiality, integrity, availability)
 - **Privacy** (data access and usage rules)
- **Definition of privacy policies**
 - FOR patients, health institutions, health professionals
 - Access rules based on roles and conditions

Example: Privacy in eHealth

- Privacy policies
 - Use of XACML standard
- High level of granularity
- Permitted actions, roles and attributes
 - **Actions**: Create, View, ...
 - **Roles**: Defined by different concepts: health professional specialty, function, ...
 - **Attributes**: Lab test order, ...

Example of XACML 3.0 policy (I)

<Policy

```
xmlns="urn:oasis:names:tc:xacml:3.0:core:schema:wd-17"
xmlns:xacml="urn:oasis:names:tc:xacml:3.0:core:schema:wd-17"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="urn:oasis:names:tc:xacml:3.0:core:schema:wd-17 http://docs.oasis-
open.org/xacml/3.0/xacml-core-v3-schema-wd-17.xsd"
xmlns:md="http://www.med.example.com/schemas/record.xsd"
PolicyId="urn:oasis:names:tc:xacml:3.0:example:policyid:3"
Version="1.0"
RuleCombiningAlgId="urn:oasis:names:tc:xacml:1.0:rule-combining-algorithm:deny-overrides">
  <Description>
    Policy for any medical record in the
    http://www.med.example.com/schemas/record.xsd namespace
  </Description>
  <PolicyDefaults>
    <XPathVersion>http://www.w3.org/TR/1999/REC-xpath-19991116</XPathVersion>
  </PolicyDefaults>
  <Target>
    <AnyOf>
      <AllOf>
        <Match>
          MatchId="urn:oasis:names:tc:xacml:1.0:function:anyURI-equal">
            <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#anyURI"
              >urn:example:med:schemas:record</AttributeValue>
            <AttributeDesignator
              MustBePresent="false"
              Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
              AttributeId="urn:oasis:names:tc:xacml:2.0:resource:target-namespace"
              DataType="http://www.w3.org/2001/XMLSchema#anyURI"/>
          </Match>
        </AllOf>
      </AnyOf>
    </Target>
```

Example of XACML 3.0 policy (II)

```
<Rule RuleId="urn:oasis:names:tc:xacml:3.0:example:ruleid:3"
      Effect="Permit">
  <Description>
    A physician may write any medical element in a record
    for which he or she is the designated primary care
    physician, provided an email is sent to the patient
  </Description>
  <Target>
    <AnyOf>
      <AllOf>
        <Match
          MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
            <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string"
              >physician</AttributeValue>
            <AttributeDesignator
              MustBePresent="false"
              Category="urn:oasis:names:tc:xacml:1.0:subject-category:access-subject"
              AttributeId="urn:oasis:names:tc:xacml:3.0:example:attribute:role"
              DataType="http://www.w3.org/2001/XMLSchema#string"/>
          </Match>
        </AllOf>
      </AnyOf>
    .....
```

Example of XACML 3.0 policy (III)

```
.....
</Target>
<Condition>
  <Apply FunctionId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
    <Apply
      FunctionId="urn:oasis:names:tc:xacml:1.0:function:string-one-and-only">
        <AttributeDesignator
          MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:1.0:subject-category:access-subject"
          AttributeId="urn:oasis:names:tc:xacml:3.0:example: attribute:physician-id"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
        </Apply>
      <Apply
        FunctionId="urn:oasis:names:tc:xacml:1.0:function:string-one-and-only">
          <AttributeSelector
            MustBePresent="false"
            Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
            Path="md:record/md:primaryCarePhysician/md:registrationID/text()"
            DataType="http://www.w3.org/2001/XMLSchema#string"/>
          </Apply>
        </Apply>
      </Apply>
    </Condition>
  </Rule>
```


Example of XACML 3.0 policy (IV)

```
.....
<ObligationExpressions>
  <ObligationExpression ObligationId="urn:oasis:names:tc:xacml:example:obligation:email"
    FulfillOn="Permit">
    <AttributeAssignmentExpression
      AttributeId="urn:oasis:names:tc:xacml:3.0:example:attribute:mailto">
      <AttributeSelector
        MustBePresent="true"
        Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
        Path="md:record/md:patient/md:patientContact/md:email"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </AttributeAssignmentExpression>
      <AttributeAssignmentExpression
        AttributeId="urn:oasis:names:tc:xacml:3.0:example:attribute:text">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string"
          >Your medical record has been accessed by:</AttributeValue>
        </AttributeAssignmentExpression>
        <AttributeAssignmentExpression
          AttributeId="urn:oasis:names:tc:xacml:3.0:example:attribute:text">
          <AttributeDesignator
            MustBePresent="false"
            Category="urn:oasis:names:tc:xacml:1.0:subject-category:access-subject"
            AttributeId="urn:oasis:names:tc:xacml:1.0:subject:subject-id"
            DataType="http://www.w3.org/2001/XMLSchema#string"/>
          </AttributeAssignmentExpression>
        </ObligationExpression>
      </ObligationExpressions>
    </Policy>
```

Example of XACML 3.0 policy

```
<Rule RuleId="urn:oasis:names:tc:xacml:3.0:RuleSAM" Effect="Permit">
  <Description> A physician may view a document about his/her patient
</Description>
  <Target>
    <AnyOf> <AllOf>
      <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
          Physician
        </AttributeValue>
        <AttributeDesignator MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:3.0:role" AttributeId="role"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </Match>
      <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:regexp-string-match">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
          my-patient-document.pdf
        </AttributeValue>
        <AttributeDesignator MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
          AttributeId="urn:oasis:names:tc:xacml:1.0:resource:resource-id"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </Match>
      <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
          View
        </AttributeValue>
        <AttributeDesignator MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:3.0:attribute-category:action"
          AttributeId="urn:oasis:names:tc:xacml:1.0:action:action-id"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </Match>
    </AllOf> </AnyOf>
  </Target>
</Rule>
```

Example of XACML 3.0 policy

```
<Rule RuleId="urn:oasis:names:tc:xacml:3.0:RuleSAM" Effect="Permit">
  <Description> A physician may view a document about his/her patient
</Description>
  <Target>
    <AnyOf> <AllOf>
      <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
          Physician
        </AttributeValue>
        <AttributeDesignator MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:3.0:role" AttributeId="role"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </Match>
      <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:regexp-string-match">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
          my-patient-document.pdf
        </AttributeValue>
        <AttributeDesignator MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
          AttributeId="urn:oasis:names:tc:xacml:1.0:resource:resource-id"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </Match>
      <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
        <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
          View
        </AttributeValue>
        <AttributeDesignator MustBePresent="false"
          Category="urn:oasis:names:tc:xacml:3.0:attribute-category:action"
          AttributeId="urn:oasis:names:tc:xacml:1.0:action:action-id"
          DataType="http://www.w3.org/2001/XMLSchema#string"/>
      </Match>
    </AllOf> </AnyOf>
  </Target>
</Rule>
```

Rule effect

Example of XACML 3.0 policy

```
<Rule RuleId="urn:oasis:names:tc:xacml:3.0:RuleSAM" Effect="Permit">
```

Rule effect

```
<Description> A physician may view a document about his/her patient
```

```
</Description>
```

```
<Target>
```

```
<AnyOf> <AllOf>
```

```
<Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
  <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
    Physician
  </AttributeValue>
  <AttributeDesignator MustBePresent="false"
    Category="urn:oasis:names:tc:xacml:3.0:role" AttributeId="role"
    DataType="http://www.w3.org/2001/XMLSchema#string"/>
</Match>
```

Authorized role: Physician

```
<Match MatchId="urn:oasis:names:tc:xacml:1.0:function:regex-string-match">
```

```
<AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
```

```
my-patient-document.pdf
```

```
</AttributeValue>
```

```
<AttributeDesignator MustBePresent="false"
```

```
Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
```

```
AttributeId="urn:oasis:names:tc:xacml:1.0:resource:resource-id"
```

```
DataType="http://www.w3.org/2001/XMLSchema#string"/>
```

```
</Match>
```

```
<Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
```

```
<AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
```

```
View
```

```
</AttributeValue>
```

```
<AttributeDesignator MustBePresent="false"
```

```
Category="urn:oasis:names:tc:xacml:3.0:attribute-category:action"
```

```
AttributeId="urn:oasis:names:tc:xacml:1.0:action:action-id"
```

```
DataType="http://www.w3.org/2001/XMLSchema#string"/>
```

```
</Match>
```

```
</AllOf> </AnyOf>
```

```
</Target>
```

```
</Rule>
```

Example of XACML 3.0 policy

```
<Rule RuleId="urn:oasis:names:tc:xacml:3.0:RuleSAM" Effect="Permit">
  <Description> A physician may view a document about his/her patient
</Description>
<Target>
  <AnyOf> <AllOf>
```

Rule effect

```
    <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
      <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
        Physician
      </AttributeValue>
      <AttributeDesignator MustBePresent="false"
        Category="urn:oasis:names:tc:xacml:3.0:role" AttributeId="role"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
    </Match>
```

Authorized role: Physician

```
    <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:regex-string-match">
      <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
        my-patient-document.pdf
      </AttributeValue>
      <AttributeDesignator MustBePresent="false"
        Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
        AttributeId="urn:oasis:names:tc:xacml:1.0:resource:resource-id"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
    </Match>
```

Resource

```
    <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
      <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
        View
      </AttributeValue>
      <AttributeDesignator MustBePresent="false"
        Category="urn:oasis:names:tc:xacml:3.0:attribute-category:action"
        AttributeId="urn:oasis:names:tc:xacml:1.0:action:action-id"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
    </Match>
  </AllOf> </AnyOf>
</Target>
</Rule>
```

Example of XAMCL 3.0 policy

```
<Rule RuleId="urn:oasis:names:tc:xacml:3.0:RuleSAM"
  <Description> A physician may view a document about his/her patient
</Description>
<Target>
  <AnyOf> <AllOf>
```

Effect="Permit">

Rule effect

```
    <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
      <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
        Physician
      </AttributeValue>
      <AttributeDesignator MustBePresent="false"
        Category="urn:oasis:names:tc:xacml:3.0:role" AttributeId="role"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
    </Match>
```

Authorized role: Physician

```
    <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:regexp-string-match">
      <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
        my-patient-document.pdf
      </AttributeValue>
      <AttributeDesignator MustBePresent="false"
        Category="urn:oasis:names:tc:xacml:3.0:attribute-category:resource"
        AttributeId="urn:oasis:names:tc:xacml:1.0:resource:resource-id"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
    </Match>
```

Resource

```
    <Match MatchId="urn:oasis:names:tc:xacml:1.0:function:string-equal">
      <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
        View
      </AttributeValue>
      <AttributeDesignator MustBePresent="false"
        Category="urn:oasis:names:tc:xacml:3.0:attribute-category:action"
        AttributeId="urn:oasis:names:tc:xacml:1.0:action:action-id"
        DataType="http://www.w3.org/2001/XMLSchema#string"/>
    </Match>
```

Authorized action: View

```
  </AllOf> </AnyOf>
</Target>
</Rule>
```

Security in applications

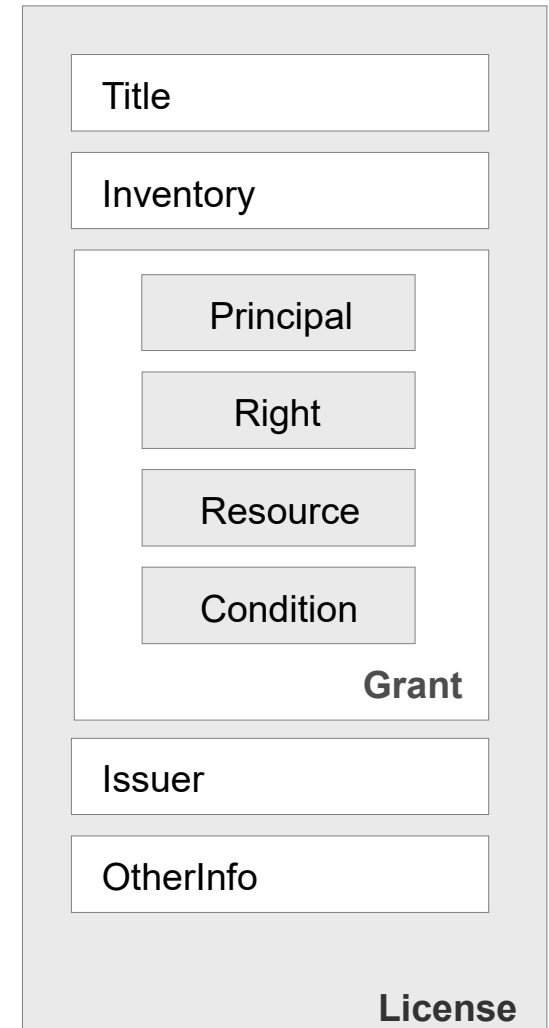
- **1.** Protocols:
 - Web (HTTP)
 - E-mail (content!)
- **2.** Formats:
 - XML: XML Encryption, XML Signature
- **3.** Protocols specific for security: SAML, OAuth, JWT
- Others:
 - **4.** Data protection / Privacy / Digital identity
 - **5. Intellectual rights for multimedia content**

Intellectual property

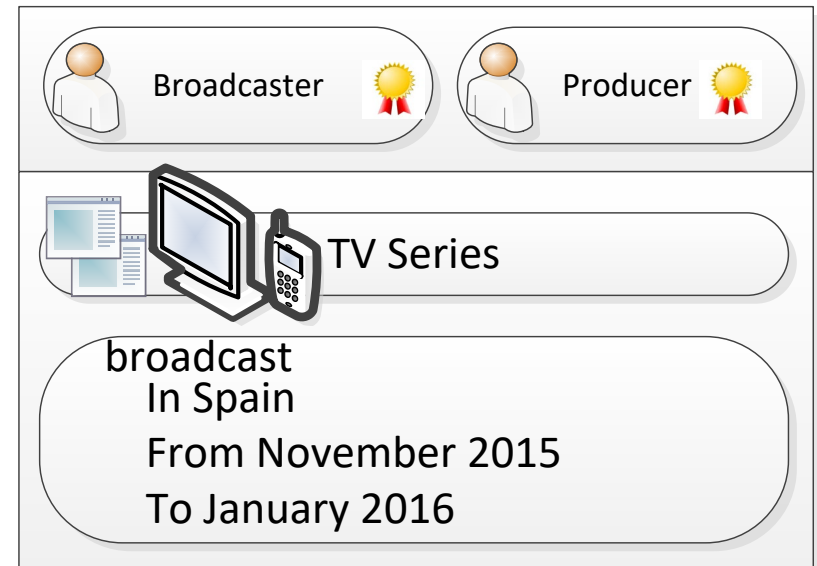
- Owned by “creators” (rights holders, *titulares de derechos*)
 - Article 27 of the Universal Declaration of Human Rights.
 - “*Ley de Propiedad Intelectual*”
 - WIPO (World Intellectual Property Organization)
 - Industrial property + Author right (Copyright)
 - Moral rights: paternity, integrity, disclosure (*divulgación*), withdrawal, ...
 - Economic rights (*patrimoniales*): reproduction, distribution, public communication, transformation
- Exclusive, Remunerated**
- Related rights (*conexos*): performers (intérpretes), broadcasters, ...

MPEG-21 Part 5: REL

- Rights Expression Language (REL)
 - Language (XML-based) for expressing rights and conditions for using content
 - Most important concept: **License**
 - <https://www.iso.org/standard/36095.html>



- Contracts Expression Language (CEL)
 - Language (XML-based) for expressing contracts defining actions and constraints for using content → Extends REL
 - Most important concept: **Contract**
 - <https://www.iso.org/standard/68926.html> (second edition)



- Use Smart Contracts to define rights over Media
- Related to CEL
 - Defines the relationship between the CEL and Smart Contract concepts
 - <https://www.iso.org/standard/82527.html>

How Multimedia Content is protected?

- Digital Rights Management Systems
 - Microsoft PlayReady
 - Intertrust ExpressPlay
 - Google WideVine
 - Apple FairPlay
 - Etc.
- Problem
 - Market segmentation
 - Storage space (one content for each DRM/platform)

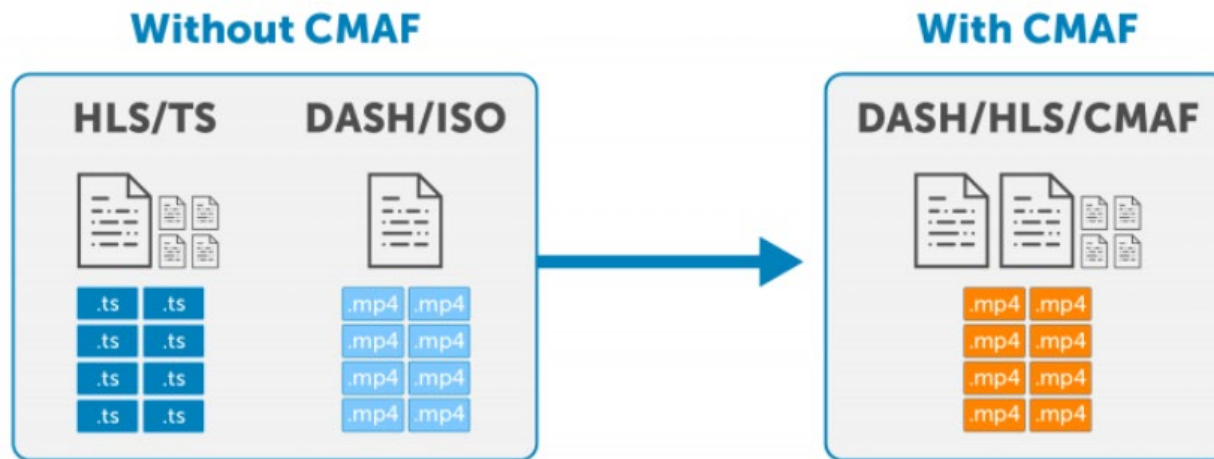
Bibliography:

<https://www.expressplay.com/blog/how-cmaf-can-reduce-multi-drm-service-costs/>

<https://aws.amazon.com/es/media/tech/speke-basics-secure-packager-encoder-key-exchange-api/>

How Multimedia Content is protected?

- Solutions
 - Use of common standardized formats and encryption protocols
 - CMAF (Common Media Application Format)
 - Simplify content packaging and distribution



- CENC (Common Encryption)
- SPEKE (Secure Packager and Encoder Key Exchange)

Source: <https://www.wowza.com/docs/about-cmaf-workflows-in-wowza-streaming-engine>

Bibliography: <https://dzone.com/articles/cmaf-common-media-application-format>

How Multimedia Content is protected?

■ What your browser can do?

Edge for Windows 10 (2021)

Decryption skills...

System	Encryption	Robustness	Supported?
Widevine	cenc	L3 - SW crypto	yes
Widevine	cenc	L2 - HW crypto	no
Widevine	cenc	L1 - HW all	no
Widevine	cbcs	L3 - SW crypto	yes
PlayReady	cenc		yes
PlayReady	cbcs		no
ClearKey	cenc		yes
ClearKey	cbcs		yes
Fairplay	cbcs		no

Decoding skills....

Codec	String	Supported?
H.264 Baseline 1.3	avc1.42000d	yes
H.264 Main 3.0	avc1.4d001f	yes
H.264 High 4.0	avc1.640028	yes
H.264 High 5.2	avc1.640034	yes
HEVC MP MT 3.1	hev1.1.6.L93.B0	yes
HEVC Dolby Vision 5.6	dvh1.05.06	no
VP9 P0 L5 8b	vp09.00.50.08	yes
AV1 MP L2.0 MT 8b	av01.0.00M.08	no
AAC-LC	mp4a.40.2	yes
Dolby Digital	ec-3	yes

Chrome for Windows 10 (2021)

Decryption skills...

System	Encryption	Robustness	Supported?
Widevine	cenc	L3 - SW crypto	yes
Widevine	cenc	L2 - HW crypto	no
Widevine	cenc	L1 - HW all	no
Widevine	cbcs	L3 - SW crypto	yes
PlayReady	cenc		no
PlayReady	cbcs		no
ClearKey	cenc		yes
ClearKey	cbcs		yes
Fairplay	cbcs		no

Decoding skills....

Codec	String	Supported?
H.264 Baseline 1.3	avc1.42000d	yes
H.264 Main 3.0	avc1.4d001f	yes
H.264 High 4.0	avc1.640028	yes
H.264 High 5.2	avc1.640034	yes
HEVC MP MT 3.1	hev1.1.6.L93.B0	no
HEVC Dolby Vision 5.6	dvh1.05.06	no
VP9 P0 L5 8b	vp09.00.50.08	yes
AV1 MP L2.0 MT 8b	av01.0.00M.08	yes
AAC-LC	mp4a.40.2	yes
Dolby Digital	ec-3	no

Bibliography: <https://websites.fraunhofer.de/video-dev/is-this-the-end-of-cenc-an-overview-of-drm-codec-support-in-2021/>
<https://research.akamaized.net/capabilitiescheck.html>

How Multimedia Content is protected?

- What your browser can do²

Firefox for Windows 10 (2021)

Decryption skills...

System	Encryption	Robustness	Supported?
Widevine	cenc	L3 - SW crypto	yes
Widevine	cenc	L2 - HW crypto	no
Widevine	cenc	L1 - HW all	no
Widevine	cbcs	L3 - SW crypto	yes
PlayReady	cenc		no
PlayReady	cbcs		no
ClearKey	cenc		yes
ClearKey	cbcs		yes
Fairplay	cbcs		no

Decoding skills....

Codec	String	Supported?
H.264 Baseline 1.3	avc1.42000d	yes
H.264 Main 3.0	avc1.4d001f	yes
H.264 High 4.0	avc1.640028	yes
H.264 High 5.2	avc1.640034	yes
HEVC MP MT 3.1	hev1.1.6.L93.B0	no
HEVC Dolby Vision 5.6	dvh1.05.06	no
VP9 P0 L5 8b	vp09.00.50.08	yes
AV1 MP L2.0 MT 8b	av01.0.00M.08	yes
AAC-LC	mp4a.40.2	yes
Dolby Digital	ec-3	no

How Multimedia Content is protected?

- What these browsers cannot do?
 - HW encryption – They are browsers...
 - Playready – cbcs
 - For Xbox One
 - Fairplay – cbcs
 - Commercial reasons (Apple)
 - HECV – dolby vision
 - Commercial reasons (Apple)
 - AV1 or Dolby Digital
 - Commercial reasons (MPEG or AV1)

Bibliography: <https://bitmovin.com/google-deprecating-widevine-drm-cdm/>

Security in Applications

Silvia Llorente
Jaime Delgado

DAC - UPC